Atomistic Simulations for Insulating Nanoporous Electrodes

Maya Rubenstein

Abstract

This study aims to develop software for modelling electrodes covered in a porous insulating layer for applications to carbon dioxide reduction. The project focuses on modular design and efficiency in its construction of these modules. It successfully produces a library for cyclic voltammogram analysis including visualization and baseline correction. It also develops a data structure for partitioning 3D space, allowing efficient collision checking. Finally, it presents designs for electrode modules that allow clients to specify the geometry and conductivity of the electrode surface.

Chapter 1

Introduction

1.1 Background and Significance

Climate change comprises one of the greatest threats currently facing humanity. Accumulation of greenhouse gases endangers air quality, human health, and energy security. Carbon dioxide (CO_2), emitted by power plants, automobiles, and airplanes, is the primary greenhouse gas. Efforts to mitigate climate change often focus on decreasing greenhouse gas emissions; however, this alone does not suffice. First, experts predict that limiting global warming to the $\Delta T < 2\,^{\circ}C$ target set by the international community in the Paris Agreement likely necessitates carbon negative techniques.[1] It is also unlikely that all significant carbon dioxide emissions can be eliminated: while electric vehicles become more common, there is no electric equivalent for airplanes. For these reasons, scientists have long been searching for ways to reduce carbon dioxide.

The most natural technique of carbon fixation is photosynthesis. While nature provides a plethora of organisms that remove carbon dioxide from the air, the efficiency of the process is limited by natural processes and seasonal variation. [1]. Photosynthetic efficiencies for plants are close to 1%, and fail to regenerate energy-dense fuel. [2] For this reason, scientists have sought to develop artificial processes that reduce carbon dioxide electrically.

Since the 1980s, copper has been known to reduce CO_2. to organic products such as ethylene, ethanol, and propanol [3] However, long-chain hydrocarbons are preferable to these C2

and C3 products due to their increased similarity to gasoline. The low volatility of loner-chain carbons makes them easier to store and transport, and their high volumetric energy density renders them a natural source of fuel. [4] In particular, 1-butanol is a desirable product as it has a similar volumetric energy density to gasoline. [8] Reducing carbon dioxide into C3+ products has proven more difficult. Calvinho and colleagues demonstrated the synthesis of the cyclic C4 product 2,3-furandiol at a faradaic efficiency of 71%. [5] Zhou and Martin showed the formation of not only C4 products, but C5 and C6 as well. However, faradaic efficiencies were low, less than 7%. [6] In 2020, Ting et al. reported the first electrochemical reduction of carbon dioxide to 1-butanol, but at FE = 0.056%, too low to be economically viable. [7] Choi et. al. improved on this, using phosphorus-rich copper cathodes to produce 1-butanol at 3.868%. [4]

In 2023, Cronin and Duvolic achieved a major breakthrough in generating 1-butanol at FE = 63%. They used a catalyst based on chromium oxide and gallium oxide in a 3:1 ratio. This mixture was dropcast onto a glassy carbon electrode, creating a surface layer. Nickel was then incorporated into the system via electrolysis. XRD analysis suggests that the chromium and gallium oxides form a nanoporous layer. [8] The catalytic activity of the system is perplexing: chromium and gallium oxides are insulators, and therefore should reasonably inhibit electron transfer from the electrode. This inverts the general advantage of nanopores, which generally bequeaths upon a system a higher surface area per volume [9]. Chromium and gallium oxides in fact limit the electroactive area, suggesting a lower faradaic efficiency. Instead, the insulators allowed production of the most viable fuel substitute in the highest faradaic efficiencies to date.

This experiment was not the first time exceptional electrochemical effects were witnessed in nanopores. Nanoelectrochemistry emerged in the 1980s and has been regularly investigated with steady-state voltammetry methods such as cyclic voltammograms. More recently, nanoelectrochemistry has exploded, and much research has focused on exploiting nanoparticles for catalysis. [10] In 2014, a comparable experiment by Sen, Liu, and Palmore explored the reduction of CO2 at nanoporous copper forms as compared to smooth copper electrodes. Like Cronin and Duvolic, they found the nanoscale environment to exhibit increased catalytic activity: formic acid was formed at faradaic efficiencies 10-20% higher than previously reported values for smooth electrodes. They also generated C3 products which were at the time novel,

again suggesting that the catalytic activity of nanopores cannot solely be attributed to increased surface area. [11]

One possible explanation for the electrocatalytic activity of the insulators in the confinement effect. It is well-known that nanoporous materials provide unique chemical environments. Unlike a planar electrode, the three-dimensional interactions of the electroactive species and the electrode come into play. The movement of the reactants is confined by the pores, increasing collisions. [12] Since mass transport is an essential component of heterogeneous reaction kinetics, limitations on movement can significantly impact the reaction. Fu et. al. used nanopore electrode arrays to demonstrate signal amplification in confined environments that arises from control of ion migration. For multi-step reactions, like the conversion of carbon dioxide to C4 products, the confinement of intermediates increases their interaction likelihood, allowing larger carbon molecules to be formed through subsequent reactions. [13]

The effect of insulating nanopores on reactivity was explored recently by Seo and Bae. They explored the catalytic effects of insulating nanoporous materials on electrochemical processes, including both single-step reactions and the more complex multi-step oxygen reduction reaction. They confirmed the catalytic effect of nanoconfinement in the absence of flat electroactive surfaces. [14] However, their work was solely experimental in nature, leaving a gap of computational modeling.

Han et al. investigated the confinement effect both experimentally and computationally in 2009. In order to confirm that the increased catalytic activity of porous material could not solely be attributed to increased surface area, they performed cyclic voltammograms and normalized for the increased surface area. They then used Grand Canonical Monte Carlo simulations to theoretically confirm their findings. They used a hard-sphere model for the electroactive species, and compared planar and random porous electrodes. Their results demonstrated that the porous electrode was likely to have the greatest catalytic effect on slow, diffusion-controlled reactions. While their system lacks the insulating components of the chromium-gallium oxide catalyst, they have demonstrated that computer simulations can show the confinement effect. [12]

However, the Han study has a significant flaw: they used Monte Carlo simulations to extract kinetic information. Monte Carlo is based off of probabilistically exploring a free energy space

and has no inherent time component. While methodologies have been developed to modify MC to extract kinetic information, these are based off of transition state theory or otherwise derive kinetic information from free energy. [16] Han et al., however, attempted to extract kinetic information directly. This does not accurately reflect the dynamics of the system: the Metropolis criterion approach means the simulation will reject displacements, so the system will not evolve in time realistically.

Our study seeks to build upon Han et al. but modify it in two crucial ways. First, instead of using an MC approach, we use diffusion directly. The Bocarsly Lab has previously used cyclic voltammograms as a powerful tool in studying carbon dioxide reduction. Since cyclic voltammograms comprise our primary source of experimental data, building CV simulations is the obvious path. Diffusion is one of the main components determining cyclic voltammograms, supporting a diffusion based-simulation model.

Second, we hope to modify the exisiting study by implementing the hypothesized geometry: pores with insulating walls and a conducting floor. The insulating components present a novel modification fof the Han et al. study. In general, we seek to build simulation software that prioritizes electrode geometry instead of solution details. Our hypothesis is that mechanical, not chemical or qunatum factors are primarily responsible for the catalytic activity of chromium and gallium. To test this hypothesis, we need a model that devotes computational power to the geometry of the electrode surface, and allows clients to be specific with their desired electrode geometry. Using a simple model of movement, like the random walk, contributes to this allocation of priority to the mechanical details of the electrode surface. This differs from molecular dynamics simulations like MetalWalls which devote much more attention to the motion of particles, and studies that use bulk parameters like tortuosity to model porous surfaces.

1.2 Specific Aims

1. Develop flexibly and reusable data structures for electrochemical modelling. We aim to build computational programs that can be used in a wide range of applications in addition to our own. We focus on modular design and efficiency: modularity involves

allocation tasks among various objects, which abstract simple calculations. Efficiency relates to computational speed. While we focus primarily on designing a framework that can evaluate our hypothesis for the chromium-gallium catalyst, we hope to produce a library that is applicable to a range of chemical systems.

2. Build electrode objects that allow clients to implement specific surface geometries. Since our hypothesis surrounds the geometry of the electrode surface, we seek to build data structures that allow users to simulate variable surface topologies. The objects should allow clients to implement specific geometries, such as defining pore sizes.

3. Simulate a cyclic voltammogram with insulating pores: This last aim involves directly testing our hypothesis regarding the catalytic activity of chromium-gallium. We hope to use our data structures to model the chromium-gallium catalyst to assess whether its increase catalytic activity can be fully attributed to mechanical considerations.

Chapter 2

Materials & Methods

We developed our simulations using Jupyter Notebooks and Python scripts accessed on Visual Studio Code. Python 3.9.6 was used. All simulations were run locally on a MacOS laptop.

In addition to custom-built modules, we used the libraries `numpy`, primarily for manipulating arrays and generating random values; `matplotlib.pyplot` for visualization; `copy` for manipulating lists; and `sortedcontainers` for efficient list manipulation.

The only experimental data collected were cyclic voltammograms for $[Fe(CN)_6]^{-4/-3}$. We used a $[Fe(CN)_6]^{-4/-3}$ concentration of 2×10^{-3} M and KCl as the supported electrolyte. We tested three different electrodes, one a standard glassy carbon electrode and two glassy carbon electrodes drop casted with a 3:1 mixture of chromium oxide and gallium oxide.

Chapter 3

Results & Discussion: Python Analysis and Baseline Correction for Ferrocyanide CVs

3.1 Introduction

The Bocarsly lab has extensively used cyclic voltammograms to study their chromium-gallium catalyst. Therefore, we began by building a Python module to analyze and visualize CVs. This module helps provide a point of comparison for simulation work. Here we display the library and using it to visualize and analyze the data for the reduction of of ferricyanide. We also present a baseline correction function.

3.2 Reading the Data into Python

First, we created a function for reading the data from the CV output file into Python. The function is quite messy and relies on (1) consistent naming convention of the files and (2) consistent formatting of the files. The function assumes one date name and one analyte (though this can be modified easily later) and loops through various electrodes and scanning speeds. It formualically creates a string equaling the filename so it can be located and imported.

```
str = date + '-' + electrode + '-' + analyte + '-' + speed + 'mvs.txt'
```

```
fileObject = open(foldername+'/'+str, 'r')
```

The function then lops through the lines of `fileObject`, checking if they match the expected fields of the CV program output file. If so, it extracts the information and saves in to the CV object.

3.3 The CV Object

We also created a class for storing the CV data. The class has attributes for various CV parameters. These include the experimental conditions (analyte, electrode, and scan rate) as well as measured variables like the peak current in each segment. The class also contains methods for analyzing and visualizing the data. The class assumes that the scan proceeds cathodically. Aside from data input methods and baseline correction methods (to be discussed later), the class methods include:

1. **Ehalf:** average Ep_1 and Ep_2, the potentials corresponding to peak current, to find the $E_{1/2}$ value and thereby estimate the standard potential of the cell.

2. **graph:** graph the CV data, given a `matplotlib` subplot object. It also automatically names the axes.

3. **represent:** create a string description of the CV. This can be used for titling graphs or creating legends.

The input function and CV class allow for easily manipulation of data. For example, Figure 3.1 shows the small amount of code needed to graph all the $[Fe(CN)_6]^{-4/-3}$ CVs.

3.4 $[Fe(CN)_6]^{-4/-3}$ Cyclic Voltammograms

We processed $[Fe(CN)_6]^{-4/-3}$ CVs with three different electrodes at eight different scan rates. Two of the electrodes are the Cr-Ga oxide electrodes being studied here, whereas the glassy carbon electrode serves as a control. Table 3.1 shows $E_{1/2}$ values for scans where Ep_1 and Ep_2 were identified using the baseline correction method discussed below.

```python
import numpy
from CVAnalysisFunctions import importCV, CV

datestrings = ['06092024']
electrodes = ['glassycarbon', 'ARE2CrGaOx','NE4CrGaOx']
analyte = 'fecn'
speeds = ['25', '50', '100', '100final', '200', '500', '750', '1000', '2000']

dataArray = numpy.ndarray((len(speeds),len(electrodes)), dtype = CV)

importCV('09062024', datestrings, electrodes, speeds, analyte, dataArray)
```

✓ 0.0s Python

```
no 06092024-glassycarbon-fecn-100mvs.txt
```

+ Code + Markdown

```python
from matplotlib import pyplot as plt
fig, ax = plt.subplots(len(speeds), len(electrodes))
fig.set_figheight(32)
fig.set_figwidth(26)
for j in range(len(electrodes)):
    for i in range(len(speeds)):
        try:
            currentGraph = dataArray[i, j]
            currentPlot = ax[i, j]
            currentGraph.graph(currentPlot)
            currentPlot.legend([currentGraph.represent()])
        except:
            continue
```

✓ 2.0s Python

Figure 3.1: The simple code needed to create the graph cyclic voltammograms.

	Glassy Carbon	**ARE2CrGaOx**	**NE4CrGaOx**
25 mV/sec	0.2105 V	0.212 V	0.2135 V
50 mV/sec	0.2105 V	0.2125 V	0.215 V
100 mV/sec		0.2085 V	0.2145 V
100final mV/sec	0.206 V	0.2105 V	0.2125 V
200 mV/sec	0.2025 V	0.209 V	0.2145 V
500 mV/sec	0.205 V	0.205 V	0.209 V
750 mV/sec	0.192 V	0.2005 V	0.208 V
1000 mV/sec	0.1935 V	0.196 V	0.2065 V
2000 mV/sec	0.179 V	0.1825 V	0.1975 V

Table 3.1: $E_{1/2}$ values for $[Fe(CN)_6]^{-4/-3}$ reduction with various electrodes at various scan rates.

Figure 3.2 displays graphs of various electrodes at the same scan rates to visually compare the Faradaic efficiency. In all scans, the ARE2CrGaOx electrode is the most effective electrode surface; the orange electrode trace always has the largest peak currents. At speeds 25-200 mV/sec, the glassy carbon control has similar peak heights to the second Cr-Ga oxide electrode, NE4CrGaOx. At the four highest speeds, 500-2000 mV/sec, the discrepency between NE4CrGaOx and the control electrode is clearer, with the NE4CrGaOx reactions showing higher peak currents than glassy carbon.

These observations elicit two questions: (1) What is different between the two Cr-Ga oxide electrodes such that ARE2CrGaOx consistently shows higher efficiency than NE4CrGaOx?

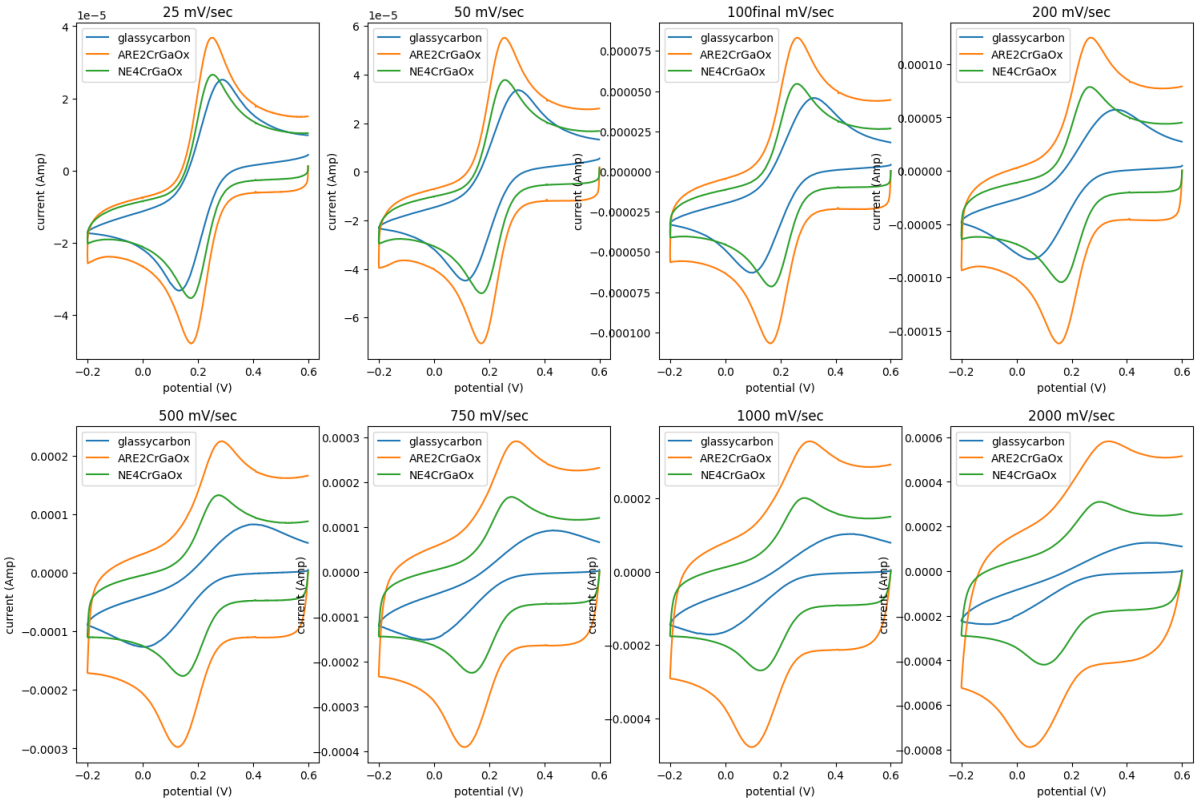

Figure 3.2: comparison of electrodes reducing ferrocyanide.

This cannot be contributed to random fluctuation in electrode assembly, because there is a higher discrepency between the two catalyst electrodes than between NE4CrGaOx and the control. (2) How do we take into account the increased catalytic advantage seen at higher scan rates?

Figure 3.3 slices the data the other way: comparing the effect of scan rate on each electrode. Figure 3.4 presents a more quantitative picture, displaying peak current as a function of scan rate (5A & 5B). The figure also shows fits to the Randles-Sevcik Equation, which proposes that max current grows as the square root of scan rate:

$$i_p = 2.69 \times 10^5 ACn^{3/2}D^{1/2}v^{1/2}$$

where n is the number of electrons transferred in the redox reaction, A is the electrode area, D is the diffusion constant, C is concentration, i_p is the peak current, and v is the scan rate. The agreement of the model to the data points for both Cr-Ga oxide electrodes, suggests that the reaction is diffusion-controlled. However, the glassy carbon depedence on scan rate does not

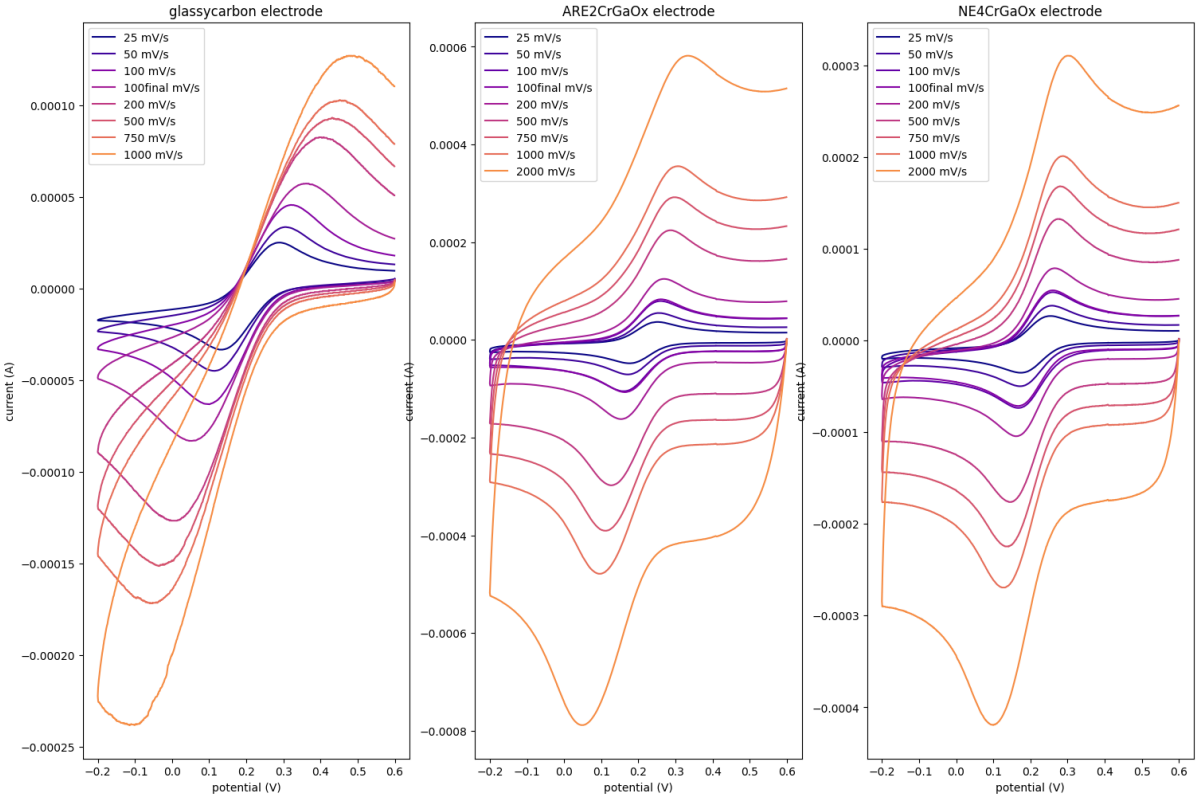

Figure 3.3: comparison of scan rates reducing ferrocyanide.

follow the $v^{1/2}$ shape. Graphing peak-to-peak separation (Figure 3.4) reveals its depedence on scan rate, suggesting that the reaction is quasi-reversibile. [1]

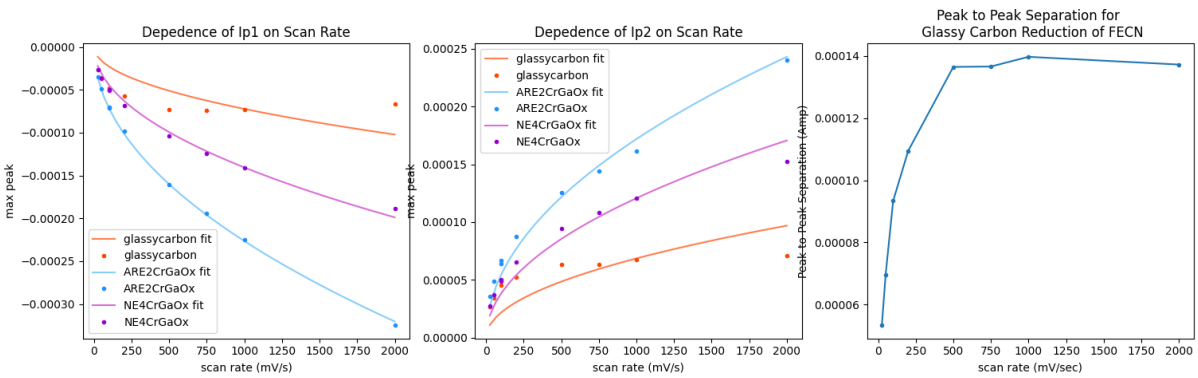

Figure 3.4: Analyzing the relationship between peak current and scan rate for $[Fe(CN)_6]^{-4/-3}$.

[1] J. Chem. Educ. 2018, 95, 2, 197–206

3.5 Baseline Correcting

As part of our analysis, we developed a strategy for performing a baseline correction on the peak current values. This is important for comparison to simulations, as the upward slope of the duck shape results from the electric double layer, which we do not simulate, meaning that our simulation can only correspond to baseline-correct data. Figure 3.5 shows the general paradaigm. For each segment, an inflection point must be located where the drop/rise of the reaction peak begins. (Note that this is not an inflection point in the literal mathematical sense; ie, where the concavity changes.) The inflection point is used in conjunction with the final point in the segment to generate a baseline current value. For each segment, this baseline is subtracted from the CV current data. The maximum/minimum current and its corresponding potential value is then determined.

The challenge here is determining the inflection points. Visually, determination of the beginning of the ”hump” is straightforward, but developing a computer algorithm to do it is trickier. we leveraged the fact that visually, the beginning of the hump corresponds to the curve's tansition from progressing mostly horizontally to mostly vertically. This again is a visual, qualitative statement that is determined by the scale used to display the graph; a horizontally elongated graph will show all regions of the curve closer to horizontal while the opposite is true of a vertically elongated graph. To circumvent this, we first normalized both the current and potential data arrays. This ensured that the quantitative values corresponded to the visual graph.

```
def normalize (self, array):
     range = max(array) - min(array)
     return [item/range for item in array]
xnorm = normalize(potential)
ynorm = normalize(current)
```

We then computed the discrete derivative of the CV segment:

$$\left(\frac{di}{dE}\right)_n = \frac{i_{n+1} - i_n}{E_{n+1} - E_n}$$

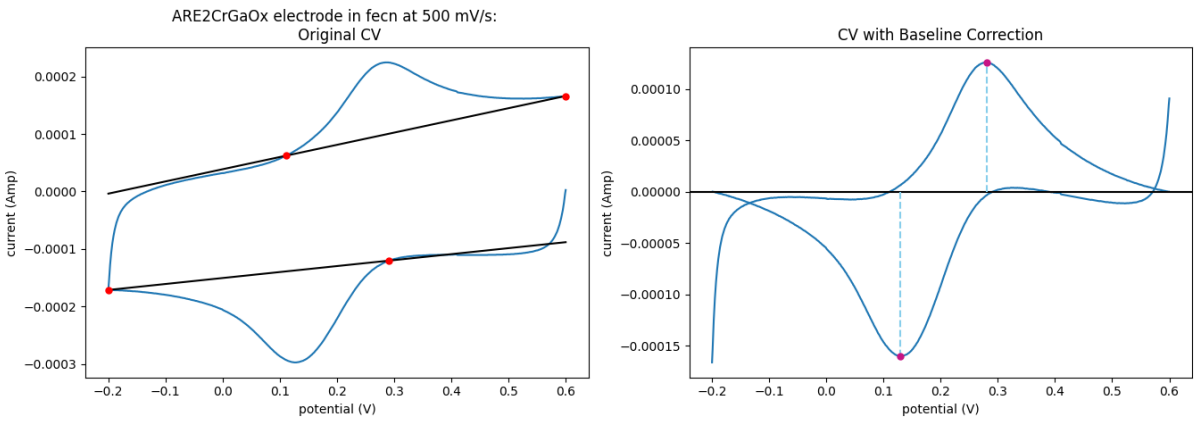

Figure 3.5: Visualization of baseline correction. Left: For each segment, an inflection point is used to find the baseline. These inflection points are shown in red, and the baseline is shown in black. Right: The baseline is subtracted from the data, and the min/max current values are found. The violet dots show the min/max currents.

After this, we smoothed the first derivate using a Savitsky-Golay filter, and then truncated the first derivative array. We first truncated by assuming that the first 100 data points in the scan would preceed the inflection point. After that, we searched for the index of the peak current and disregarded all data after the point, since the hump must necessarily start before i_p is attained.

The key step was searching for where the derivative equals one. If a curve is mostly progressing horizontally, than the absolute value of its derivative is less than 1:

$$\left|\frac{di}{dE}\right| < 1$$

whereas, if the graph is progressing vertically, the absolute value of the slope is greater than one. This technique allowed identification of the inflection point.

This process is not perfect. First, the $\frac{di}{dE} = 1$ criterion is problematic. If, say, the slope begins around 1.5 and then drastically changes to approximately 5, this qualitatively comprises the start of the hump, but the algorithm would not indetify it. Second, random oscillations in the derivative due to irregularities could cause a false positive to be identified. Third, truncating the

first 100 data was somewhat arbitrary and based on this particular data set. It might not apply to all CV scans.

Figure 3.9 shows the experimenally determined inflection points for the ferrocyanide CV's. The algorithm does not find the inflection points completely accurately, but successfully identifies the region for all grpahs following the characteristic duck shape.

3.6 Assessing the Baseline Correction Method

First, we compared the current values from my algorithm with those generated by the CV program. Table 3.2 compares i_p values from the program, my own baseline correction, and by simply taking the max current. In several cases, my algorithm was able to perform a baseline correction where the CV program failed. However, this does not provide information about the accuracy of the method.

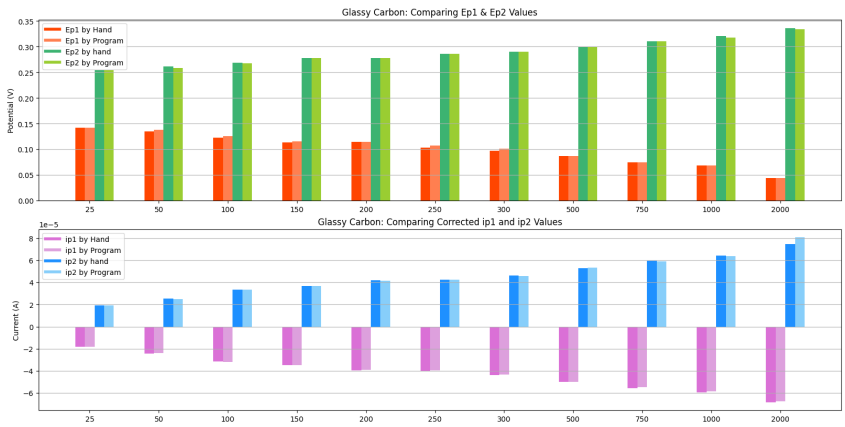

Figure 3.6: Comparing i_p and E_p values for the reduction of $[Fe(CN)_6]^{-4/-3}$ by the glassy carbon done by hand (darker colors) and by the algorithm (lighter colors.) All the values show close agreement, with discrepencies tending to grow for faster scan rates.

To better provide accuracy, we used a pre-analyzed dataset to compare i_p values derived via baseline correction done by hand with those calculated by my algorithm. Figures 3.5-3.8 show these comparisons. Figure 3.8 shows percent error, which was calculated by taking the hand-analysis values as expected. For both electrodes, all the E_p values show high agreement, with deviations under 5%. Most i_p values had similarly small errors, with two exceptions. One exception was the glassy carbon scan done at 2000 mV/sec. Still, the algorithm reproduced

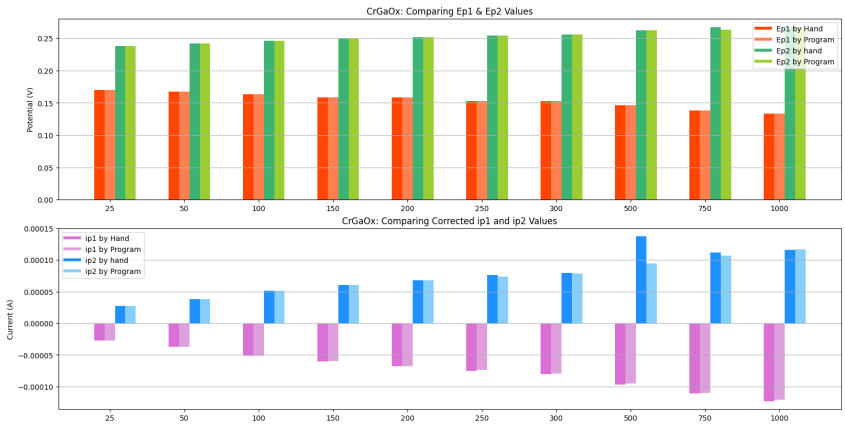

Figure 3.7: Comparing baseline correction done by hand and by the algorithm for the reduction of $[Fe(CN)_6]^{-4/-3}$ with the Chromium Gallium oxide electrode. With the exception of the current value at a scan rate of 500 mV/sec, all vlues show high agreement.

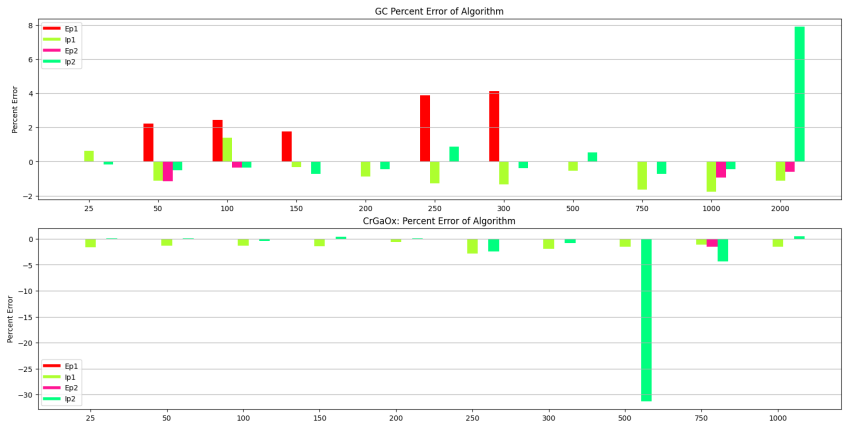

Figure 3.8: Percent error calculations for the baseline correctin algoirthm done by taking the hand-analyzed values as the expected values. All E_p values have error under 5%. With two outliers, all i_p values also have erros under 5%.

the value within an error of 8%. The other outlier is the maximum current for the CrGaOx electrode at a scan rate of 500 mV/sec. This error was much larger, with a magnitude of over 30%. We prpose that this error stems human error in the analysis.

Scan Rate (mV/sec)	Source of i_p	glassycarbon electrode	ARE2CrGaOx electrode	NE4CrGaOx electrode
25	CV Program	2.84e-05	3.95e-05	3.06e-05
	My Baseline Correction	2.65e-05	3.53e-05	2.75e-05
	Max Current	2.52e-05	3.68e-05	2.66e-05
50	CV Program	3.48e-05	5.44e-05	4.15e-05
	My Baseline Correction	3.42e-05	4.85e-05	3.7e-05
	Max Current	3.37e-05	5.52e-05	3.78e-05
100	CV Program		7.18e-05	5.49e-05
	My Baseline Correction		6.42e-05	4.87e-05
	Max Current		7.96e-05	5.26e-05
100final	CV Program	4.16e-05	7.36e-05	5.55e-05
	My Baseline Correction	4.53e-05	6.63e-05	5e-05
	Max Current	4.57e-05	8.32e-05	5.45e-05
200	CV Program		9.8e-05	7.36e-05
	My Baseline Correction	5.12e-05	8.77e-05	6.53e-05
	Max Current	5.74e-05	0.0001249	7.87e-05
500	CV Program		0.0001348	0.0001035
	My Baseline Correction	6.22e-05	0.0001255	9.4e-05
	Max Current	8.26e-05	0.0002244	0.0001323
750	CV Program		0.0001478	0.0001179
	My Baseline Correction	5.87e-05	0.0001436	0.0001081
	Max Current	9.33e-05	0.000292	0.000168
1000	CV Program		0.0001502	0.0001279
	My Baseline Correction	5.9e-05	0.0001594	0.0001204
	Max Current	0.000103	0.0003557	0.0002009
2000	CV Program			0.0001456
	My Baseline Correction	5.93e-05	0.0002355	0.0001515
	Max Current	0.0001271	0.0005816	0.0003103

Table 3.2: Peak current values for the upper segment of the ferrocyanide CV taken from the CV program, my own baseline correction, and by simply searching for the max value of the current.

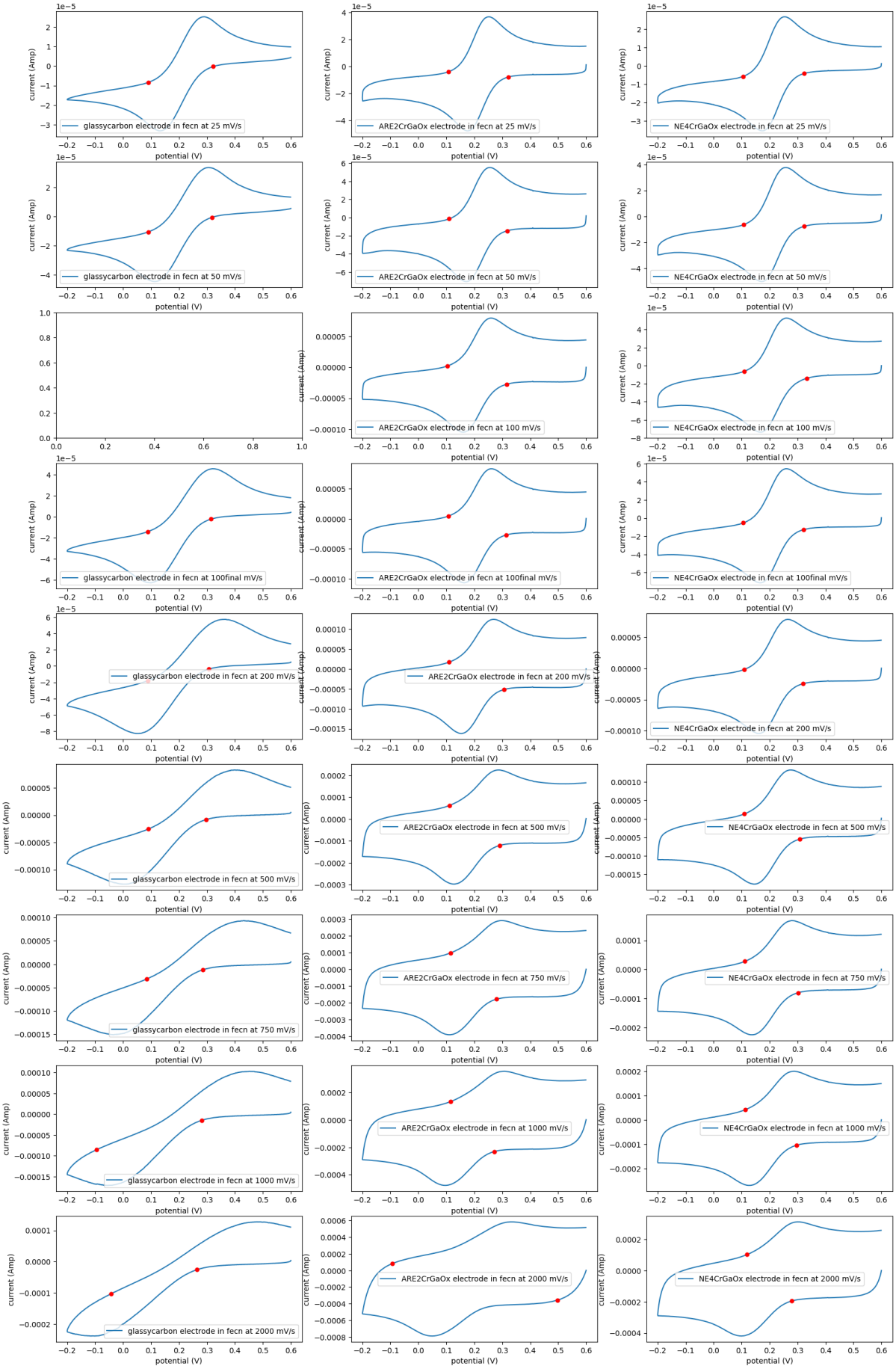

Figure 3.9: The algorithmically determined infelction points for CV scans of ferrocyanide reduction.

Chapter 4

Basic Simulation Schematic

4.1 Introduction

Cyclic voltammograms are controlled primarily by two processes: diffusion and kinetics. In this chapter I begin to introduce the framework of the experiment, and how I integrate these two key components of electrochemical reactions. I draw on Han et. al. as well as *Electrochemical Methods: Fundamentals and Applications*, by Bard and Faulkner. We begin by designing our simulation to model the simple $[Fe(CN)_6]^{-4/-3}$ reaction, where we are looking at a single pair of reactant and product.

4.2 Modelling Diffusion

According to probability theory, the displacement of a particle undergoing a random walk with variance σ^2 and mean 0 can be modelled by a random variable $N(0, \sigma^2 t)$ for any time t, where N is the Normal or Gaussian distribution. Random walks uphold independent increments, so we can model each timestep individually. By Einstein's' relation, the variance is given by $\sigma^2 = 2D$, where D is the diffusion constant. This allows us to iterate our system by moving each particle by a distance given from this distribution at each time step.

While the above equation tells us how to move a particle in each iteration, the finite-size effects of our simulation box pose an additional difficulty. We are only modelling a microscopic volume of the solution; computational complexity prevents us from modelling a larger

section. In a real solution, reactant and product species change in a local environment around the electrode as redox occurs; however, far from the electrode, concentrations remains at their original values. Han et al. modelled this by dividing the simulation cell along the z-axis into a reaction region and a bulk solution region, and adding or deleting particles at each time step to keep the bulk region constant at the intitial concentration. This is the approach we will use.

However, we must set the size of the reaction region to be tall enough such that this constant concentration assumption is valid. We propose that to do this, we choose the z-length of the region such that it is unlikely for a particle to diffuse across its entirety over the course of the simulation. This ensures that any product created at the elctrode cannot enter the bulk region, and thus the bulk concentration is indeed uneffected by the reaction. Suppose that we run our reaction for M timesteps δt for a total time of T:

$$T = M\delta t$$

Then, the probability distribution of the particle's displacement is

$$N(0, \sigma^2 T)$$

If we choose to set the z-length to be two standard deviations above the mean, then there is a less than 97% chance the particle will travel this distance in the time of the simulation. Thus, we let the z-length of the reactant box be

$$L_{z,\text{rx}} \geq 2\sqrt{2DT}$$

Han et al. used a 4:1 ratio for the z-length of the reactant to bulk region to bulk region, which we will mimic here. Thus, the bulk region will have height

$$L_{z,\text{bulk}} \geq \frac{1}{2}\sqrt{2DT}$$

.

While one face of the simulation box represents the electrode, we have to account for dif-

fusion out of the other five faces. In the x and y directions, we will use standard periodic boundary conditions: a particle that diffuses out the positive x side of the box reappears in the negative x side, and vice versa:

```
def pbc(self, vector, lengths):
  newVector = numpy.zeros(len(vector))
  for i in range(len(vector)):
     coord = vector[i]
     while coord < -lengths[i]/2:
       newVector[i] = coord + lengths[i]
       coord += lengths[i]
    while coord > lengths[i]/2:
      newVector[i] = coord - lengths[i]
      coord -= lengths[i]
    else:
      newVector[i] = coord
  return newVector
```

For the top of the simulation cell, we randomly create a new reactant molecule in the bulk region whenever a particle exits the cell. This helps maintain the bulk concentration as constant.

4.3 Reacting at the Electrode

Han et al. modeled the reaction as follows: if a reactant particle stayed longer than a cutoff time within a certain distance of the electrode, it was removed with probability 1. We propose to model the reaction slightly differently: if a particle collides with the electrode (defined as the radius making contact with the electrode surface), then the particle transforms into a product particle with a given probability. This probability is based on the overpotential and the rate constant.

4.4 Timestep

Next, we need to choose an appropraite timestep for the system. Generally, timesteps should be less than 1/10 th of the shortest timescale inherent in the system. Here, the parameters involving time are the diffusion constants, both about $7 \times 10^8 \text{nm}^2/\text{sec}$, and the rate constant, 1×10^{-3} Hertz. Inverting them gives 1×10^3 seconds and 1.43×10^{-9} sec/nm^2. Thus we propose to use 1×10^{-10} for our timestep.

4.5 Collisions

Monte Carlo and Molecular Dynamics simulations generally employ potential energy models to predict chemical behavior. Here, we forgo such models, and allow particles to move according to diffusion. However, we do have one restriction: molecules cannot occupy the same space. If a new position is proposed by the random generator, but moving there would mean coinciding with another molecule, the move is rejected. In this sense, molecules are imagined as hard spheres.

Technically, this formalism corresponds to the Metropolis criteria with the simple potential energy landscape

$$E(p_1, p_2) = \begin{cases} \infty, & \text{particles overlap} \\ 0 & \text{otherwise} \end{cases}$$

However, this differs from a Monte Carlo simulation because (1) time is inherent to the system as determining the displacement variance and (2) we expect that collisions among particles will be relatively rare, such that the overall motion of particles is controlled by diffusion. This renders the model is more kinetically accurate than the MC simulation of Han et al.

In the chapters that follow, it will be clear that collision checking is slow and arduous. However, efficient collision checking is an essential aspect of this model. This is because collisions among particles are what allow particles to react with each other. In the simple $[\text{Fe(CN)}_6]^{-4/-3}$ we don't expect any particle-particle interactions–particles only react at the electrode surface–

so collision checking is essentially a simplistic Metropolis criterion. However, the ultimate goal of this study is to build a simulation procedure that can be used to investigate the reduction of carbon dioxide. Building C4 products necessarily involves reactions between multiple CO_2 particles. Data structures that carefully detect collisions allow easy substitution of particle-on-particle reactivity.

4.6 The Molecule Object

Finally, we introduce our first custom simulation object: `molecule`. This module represents molecules. So far, we have only built an object for $[Fe(CN)_6]^{-4/-3}$, although extension to more complicated sets of chemical species is straightforward.

The `molecule` class contains the following instance variables:

1. `pos`: the molecule's position
2. `species`: a string denoting the chemical species of the molecule
3. `rad`: the molecule's radius, modelled as a hard sphere
4. `diffusion`: the diffusion coefficient of the molecule at 25 $^\circ$ C

The object contains a single public method: `react()`. `react()` uses a random number generator along with the rate constant and overpotential to determine if the particle will react. If the particle reacts, helper methods update the instance variables species, rad, and diffusion.

The object also includes comparator methods that tell data structures such as `sortedcontainers`'s `SortedList()` to compare `molecule` objects based on address. This helps maintain the identity of the object throughout the simulation, even when position and species change.

Chapter 5

Partititioning the Space: Volume Data Structures

5.1 Introduction

To modularize the simulation, we built a data structure, volume Hash Table, to handle 3D space. Hash tables, or symbol tables (implemented in Python as the `dict` class) allow for simple and quick lookup of values through their keys. Theoretically, to quickly add and delete particles in the simulation, we could use a particle's coordinates as the key and a `particle` datatype as the value. The issue with this is that the hash table allows us to search for a particle at an exact coordinate, but it does not have any functionality for checking ranges; for example, if we want to know if there is a particle at position (0, 1, 2), it is easy to check in a dictionary. But if we want a list of all particles within a distance of (0.5) form (0, 1, 2), the dictionary does not help us.

VolumeHT solves this by using a dictionary to save not coordinate-particles pairs, but to partition the space. This way, we don't have to check every single particle in the system, but rather only those in adjacent boxes. We give `VolumeHT` an `adjacent` method that takes a box as input and returns a list of the adjacent boxes. The boxes are numbered, and this (x, y, z) numbering saved as a tuple is used for the keys in the dictionary.

5.2 Boxes

The volume hash table depends on another data type: `Box`, which represents the compartmentalized boxes that comprise the overall simulation box. The primary functionality of the `Box` object is to store the particles contained within. For this purpose, it has methods for adding and deleting particles as well as for returning the total population (see Table 5.1). We present two different `Box` variants that implement this same basic functionality.

5.2.1 `radiusBox`

In addition to basic particle storage methods, `radiusBox` contains methods that `volumeHT` calls for collision checking. These include `allowedMoveInBox()`, which checks collisions between a given particle and other particles in the box; `allowedMoveOutOfBox`, which checks if a particle in another box might overlap with particles in this box. This can occur because our particles have nonzero radius: particle positions are really the positions of their center coordinate, but if a particle is less than the length of its radius away from the edge of a box, it can potentially occupy two boxes at once. A particle whose center is located at the corner of a box can overlap with eight boxes at once. Our data structure is built such that a particle can only be saved in one box at a given time, but we must be careful to check for collisions.

`class radiusBox`	
`int population`	returns the number of particles in the box
`void add(item)`	adds an item to the Box
`void delete(item)`	delete a given item from the Box
`boolean isEmpty`	checks if the box is empty
`list particles`	returns a list of the `molecule` or `particle` objects in the box
`fromCenter(coord, radius)`	evaluates if the particle is in the center sphere of the box (see below)
`allowedMoveInBox(item, coord)`	checks if `item` would overlap with other partilces in the same box if moved to position `coord`
`allowedMoveOutOfBox(item coord)`	checks if `item`, if moved to a position `coord` in a different box, would overlap with particles in this box

Table 5.1: API for the `Box` class used in `VolumeHT`

5.2.2 The `fromCenter()` Method

Even though the `volumeHT` approach limits the particles we must check for collisions, the checking function is still slow. We are working in 3D space: this means that a given box B has 26 adjacent boxes. A particle centered in B could potentially collide with particles in any of these boxes.

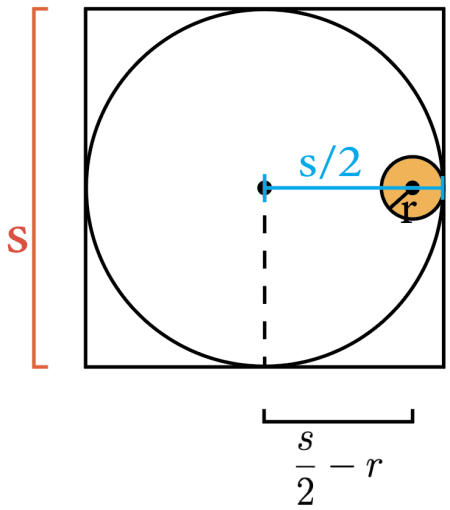

Figure 5.1: If a particle if more than the length of its radius away from teh edge of a circumscribed circle, it cannot overlap with particles in other `Box`es.

For this reason, we add the `fromCenter()` method to our `Box` classes. Each `Box` object will save its center coordinate as an object variable. As we can see in Figure 5.1, if s is the side length of the boxes and r is the particle's radius, a particle whose center is at a distance of less than $\frac{s}{2} - r$ from the center of the box can never be outside the box (Figure 5.1 shows the two dimensional case but it is easy to see that this would generalize to 3 dimensions.) Thus we can add a method to check if the particle is within $\frac{s}{2} - r$ from the box's center, and therefore whether checking the particles of the 26 adjacent boxes is needed. In our actual simulation, the side lengths in each dimension won't be exactly equal, so we take s to be the smallest side length.

Another optimization we use is storing each `Box`'s particles in a `SortedList()` instead of a regular Python `list`. Because Python's `list` class is not sorted, deleted particles takes linear time. `SortedList()`, allows removal of particles in $O(\log n)$.

5.2.3 `overlapBox`

While collision checking is an essential feature of our simulation, it is not necessary for the bulk region of the simulation. Indeed, wasting time checking the bulk region for cillisions would only serve to slow the simulation. For this reason, we implement an `overlapBox` variant that allows particles to occupy the same space. While the API is the same so that clients can use them interchangeably, the allowed move methods always return True.

5.3 Partitioning the Space

Beyond the `Box` class, the essential feature of `volumeHT` is dividing the 3D space of the simulation cells. This leads to two questions: (1) the shape of the boxes and (2) the number of boxes. For the former, there are two natural options: to fill the space with cubes or to fill space with rectangular prisms similar to the overall volume in their dimension. Here, we choose to use cubes, as these work best with the `fromCenter()` method, as cubes maximize the ratio of the volume in the circumscribed sphere to the volume of the cube.

For the number of boxes, we choose a target density d of particles/box. This should not be confused with the concentration, c, of particles/volume. We solve first for box size, s, given c and d:

$$d = \frac{\text{particles}}{\text{number of boxes}} = c\left(\frac{\text{volume}}{\text{number of boxes}}\right) = cs^3 \implies s = (d/c)^{1/3}$$

Since we want an integer number of boxes in each dimension, we use

$$b_x = \text{round}\left(\frac{x}{\sqrt[3]{\frac{d}{c}}}\right)$$

where x is the length of the box along the x axis, and find b_y and b_z analogously. We need an integer number of boxes, but since we are constrained to use cubes, it is impossible to fill the

space with exact cubes. Because of this, we compute the real side lengths in each dimension by

$$s_{x,\text{real}} = x/b_x$$

and likewise for y, z.

5.3.1 Object Methods & Variables for `volumeHT`

`object variables volumeHT`	
`boxType`	the type of `box` object used
`tuple spaceCoordinates`	coordinates of the simulation box in space $(x_{min}, x_{max}, y_{min}, y_{max}, z_{min}, z_{max})$
`tuple lengths`	lengths of the dimensions of the overall simulation space, (L_x, L_y, L_z)
`tuple numberOfBoxesPerDim`	the number of boxes in each dimension, (b_x, b_y, b_z)
`tuple sideLengths`	saves the actual side lengths for each box, ie,the box dimensions, as (s_x, s_y, y_z)
`float conc`	particles/(total volume)
`float density`	number of particles/box
`dict boxDict`	Python `dict` that saves the tuple enumeration of the box as the key and the box object as the value

Table 5.2: Object variables for `VolumeHT`

Table 5.3 shows simulation methods for `volumeHT`; that is, methods used in running the simulation. For modularity, the methods included here are quite generic; `volumeHT` is meant to be a data structure that can be used in various types of simulations. For this reason, the client has total control over the `spaceCoordinates`: while our simulation aassumes that the electrode is located at $z = 0$ and the simulation cell extends in the positive z direction, the data structure can represent and partition space of any coordinate. The simulation methods include `put()`, `delete()`, `inBox()` and `attempMove()`, as well as the helper methods `coordsToBox()` and `adjacent()`.

class volumeHT	
numpy array coordsToBox(coords)	returns the box number containing the given coordinates
boolean inBox(coord)	returns True if the coordinate is in the volumeHT False otherwise
list getSurfaceParameters (dimension, direction, coord)	returns the instance variables sideLengths, spaceCoordinates, and numberOfBoxesPerDim for use in areaHT (see chapter "Electrode Modules")
list adjacent(box)	returns a list of the boxes adjacent to the given box
void put(item, coords)	adds the item to the box containing the given coordinates, checking for overlap
void delete(item, coords)	deletes the item from the box containing the given coordinates
list contents	returns a list of all items in the hash table
boolean attemptMove(item, newPos)	attempt to move the partilce item to the position newPos

Table 5.3: API for VolumeHT: Simulation Methods

Collision checking takes place in the put method, which takes as a parameter at item (ie a particle). It first ensures that the particle's coordinate actually lies in the volumeHT; by calling inBox(). It then extracts the appropriate Box object using coordsToBox. If the box type is overlapBox, the method proceeds to simply add the item to the box and then returns True. However, for radiusBox, the procedure is somewhat more complicated. put() first uses the Box's allowedMoveInBox method to check for collisions with other particles in the Box. If a collision occurs, put() returns False. However, if no collision occurs, the method cannot terminate without ascertaining that no collisions occur with particles in adjacent boxes. It first uses the fromCenter() method to query if looping through all 26 adjacent boxes in necessary. If fromCenter() returns True, put() can add the particle and return True. If fromCenter() returns False, then put() calls volumeHT's adjacent() method and calls allowedMoveOutOfBox() on each of these boxes.

attemptMove() tries to move a particle to a new position newPos. When the method is called, particle.pos is still saved to its old coordinate. Instead of rewriting the collision

checking procedure, `attemptMove` modifies the particle's position parameter to be the new position, deletes the particle from the hash table, and calls `put()`. If `put()` returns True, `attemptMove()` can terminate and return True. If it returns False, `attemptMove()` resets the particle's position and readds it to its original `Box`. In this case, we don't call `put()`, as we don't have to check for collisions; since the particle has just been occupying that space, we know that it is allowed to be there.

`attemptMove` must delete and readd the particle in order to rehash it: since the particle is assigned to a box based on position, changing the position can potentially change the appropriate box. While we could instead check if the original and final boxes are equal, we choose this version for simplicity.

5.4 Testing

We tested all the methods in both `box` classes, as well as all the methods in `volumeHT`. All methods work as expected: `volumeHT` successfully partions the space into as prisms that approach cubes, particles cannot be inserted where they coincide with other particles, rehashing is successful, and `fromCenter` works as expected. In order to thoroughly test the objects, we wrote checker and visualization functions. The visualization functions used `pyplot`. The success of `volumeHT` is exciting because it comprises a flexible data type applicable to many simulations.

Chapter 6

Electrode Modules

6.1 Introduction

This chapter centers on the main thrust of my project: how to model a porous electrode. In keeping with our focus on modularity, I designed two seperate electrode modules that can be easily interchanged in the simulation code. The first, `electrode2D`, represents a flat, conductive electrode, such as glassy carbon. The next, `porousElectrode`, models a porous electrode. It relies on two more moedules in turn: a `pore` module and a 2-dimensional analogue of `volumeHT`. This chapter focuses heavily on geometry, and how to efficiently check for relevant geometric conditions.

6.2 The Flat Electrode

This module, modeling a flat electrode, must achieve two goals. First, it handles the reactivity of the electroactive species. While reactivity is integrated into the `molecule` object, the electrodule module determines when `react()` is called. Second, and perhaps more difficult, is the effect that the electrode has on the motion of a particle.

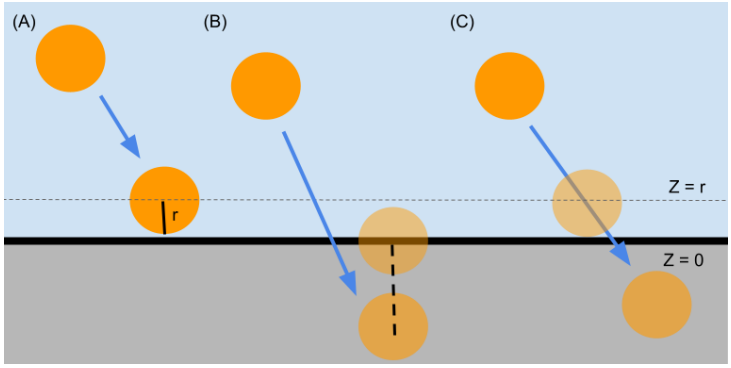

Figure 6.1: The interception of the particle with the electrode surface

6.2.1 Electrode Interception

The first aspect of this is obvious: a particle can't move into the volume occupied by the electrode. If, as we presume in our modules, the electrode surface sits at $z = 0$ and the solution zone is at $z > 0$, a particle is unable to occupy any position with $z < 0$. (See Figure 6.1(A)) So at first glance we might think we can do something like the following:

$$X, Y, Z \sim \mathrm{N}(0, \sigma)$$

$$\Delta \boldsymbol{p} = (X, Y, Z)$$

$$\boldsymbol{p}_{\text{new}} = \boldsymbol{p}_{\text{old}} + \Delta \boldsymbol{p}$$

$$\text{if } \boldsymbol{p}_{\text{new, z}} < 0 \rightarrow \text{ set } \boldsymbol{p}_{\text{new, z}} = 0$$

where $N(0, \sigma)$ denotes a normal distribution centered at 0 with standard deviation σ, taken from diffusion theory, and p is the position. However, this approach has two issues. First, we have to account for the nonzero radius of the particles. The above alogrithm ensures that the particle's *center* is not below the electrode surface; however, it does not keep the particle's volume from intersecting with the electrode volume. This introduces the modification:

$$\text{if } \boldsymbol{p}_{\text{new, z}} < r \rightarrow \text{ set } \boldsymbol{p}_{\text{new, z}} = r$$

with r the particle's radius. However, readjusting only the z-coordinate does not accuratly model how a particle moves. It will intercept the electrode along its path, which cannot be

guaranteed to be purely vertical (see Figure 6.1 (B) and (C)). Thus, instead of changing only the z-coordinate, we must find the (x, y) coordinates along the particle's path that correspond to $z = r$.

To accomplish this, we use the parametric equation of lines. For clarity, let $\boldsymbol{a}$ be the particle's original position and $\boldsymbol{b} = \boldsymbol{a} + (X, Y, Z)$ be the randomly selected new position above. We can parametrize the line by: [1]

$$\boldsymbol{l}(t) = \boldsymbol{a} + (\boldsymbol{b} - \boldsymbol{a})t = \begin{cases} a_x + (b_x - a_x)t \\ a_y + (b_y - a_y)t \\ a_z + (b_z - a_z)t \end{cases} \quad , t \in (0, 1)$$

Thus, to find $(x, y)\big|_{z=r}$, we solve

$$a_z + (b_z - a_z)t = r \implies t = \frac{r - a_z}{b_z - a_z}$$
$$\boldsymbol{p}_{\text{interception}} = \boldsymbol{a} + (\boldsymbol{b} - \boldsymbol{a})\left(\frac{r - a_z}{b_z - a_z}\right)$$

6.2.2 Rebound

Once we have found the interception point, we can attempt to react the molecule using its rate constant and the overpotential. Again, from the perspective of the electrode module, this simply amounts to call `react()` on the particle. Then, the molecule will leave the electrode, but just as at its entry, its motion is restricted. We cannot call $Z \sim N(0, \sigma)$ as before, because the particle cannot move into $z < 0$. Therefore, we modify the displacement function for particles leaving the electrode surface using polar coordinates.

The key component of this algorithm is θ, which is chosen by the client as a parameter to the electrode object. θ, which is restricted to the interval $[0, \pi/2)$, denotes the maximum angle that we allow an outgoing particle to make with the normal line to the surface (see Figure 6.2). By limiting the angle this way, we ensure that the particle is always moving in the positive z

[1] This can be written more concisely by plugging in the random variables for $a_i - b_i$; ie, $(b_x - a_x) = X$; however, we use $\boldsymbol{a}$ and $\boldsymbol{b}$ here because these are the parameters we supply our functions.

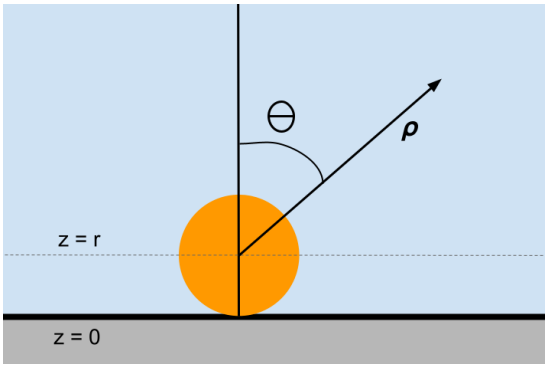

Figure 6.2: Displacing particles from the electrode surface using polar coordinates

direction. A rebounding particle selects its outgoing normal angle according to the uniform distribtion:

$$\theta(\text{particle}) \sim \text{Unif}([0, \theta])$$

Next, the angle ϕ, which determines the displacment along the $xy - plane$, is chosen according to Unif($[0, 2\pi]$) (ie equally likely any angle around the circle). Finally, the magnitude of the displacement vector, ρ, is chosen according to

$$\rho \sim \sqrt{3}\,|N(0, \sigma)| = |(N(0, \sigma), N(0, \sigma), N(0, \sigma))|$$

where the $\sqrt{3}$ makes the magnitude equivalent to the normal displacement algorithm $X_i \sim N(0, \sigma)$. Then, $\Delta\boldsymbol{p}$ is determined using the standard polar equations:

$$\Delta\boldsymbol{p} = \begin{cases} x = \rho \sin\theta \cos\phi \\ y = \rho \sin\theta \sin\phi \\ z = \rho \cos\theta \end{cases}$$

Note that we decided not to implement any algorithm that models an elastic collision: the normal angle is chosen at random and not to align with the incoming normal angle. There are two reasons for this. First, if we are modelling attraction between the particle and the electrode, then an elastic collision would be unrealistic. The particle is likely to lose momentum, especially if

it becomes stuck to the electrode. In the case of reaction, changes in atomic configuration can completely alter momentum. The second reason is that having a random algorithm captures the randomness of solution. Solvent and electrolyte molecules are not explicitly modelled here, but they are always colliding with the electroactive species and altering their path. For this reason, random restricted displacement serves as an appopriate design choice.

6.2.3 Stick

We also must account for the affinity between the reactant and the electrode surface. First of all, charged electrons create electric fields that attract ions. Beyond this, some electrode materials, in particular transition metals, are known to bind with reactants. This means that particles that collide with the electrode surface are likely to stay there for some time.

We model this with `stick` parameter in the electrode module. When an electrode hits the surface, it binds with surface, which is implemented by adding the molecule object to a `stuck` array. In each iteration, the electrode object attempts to react the particle, and then randomly determines if the particle is released to the solution or remains attached to the electrode. It determines whether the particle is to be released using

$$\text{release} \sim \text{Bernoulli}(1 - \texttt{stick})$$

This means that if `stick` is larger, the particle is more likely to stay stuck to the surface for longer (and therefore has more opportunity to react) whereas if `stick` is smaller, the particle will likely detach and return to the solution. In the limit that $\texttt{stick} = 0$, release is always equal to Bernoulli(1), which is nonrandom and always returns 1, meaning the particle always bounces off the electrode.

The `stick` parameter is not only advantageous in modeling reactant - electrode affinity. It also helps us differentiate between different surfaces in heterogenous electrodes such as those currently being developed for CO_2 reduction. Second, having particles stick to the reactant surface increases simulation speed. This is because particles are more likely to react when they have more iterations close to the electrode surface.

6.2.4 Interfacing Among Modules

The electrode object is deisgned such that it can be slipped in as a component in the outer simulation object. The electrode object must be initialized alone outside of the simulation, and then passed in as a parameter to the simulation object; the simulation will not initialize the electrode object. The electrode object does not have access to `volumeHT`, its instance variables, or its methods. Only the outer simulation object can access this functionality, including the collision checker. For this reason, clarifying the outer simulation's integration of the two modules is crucial.

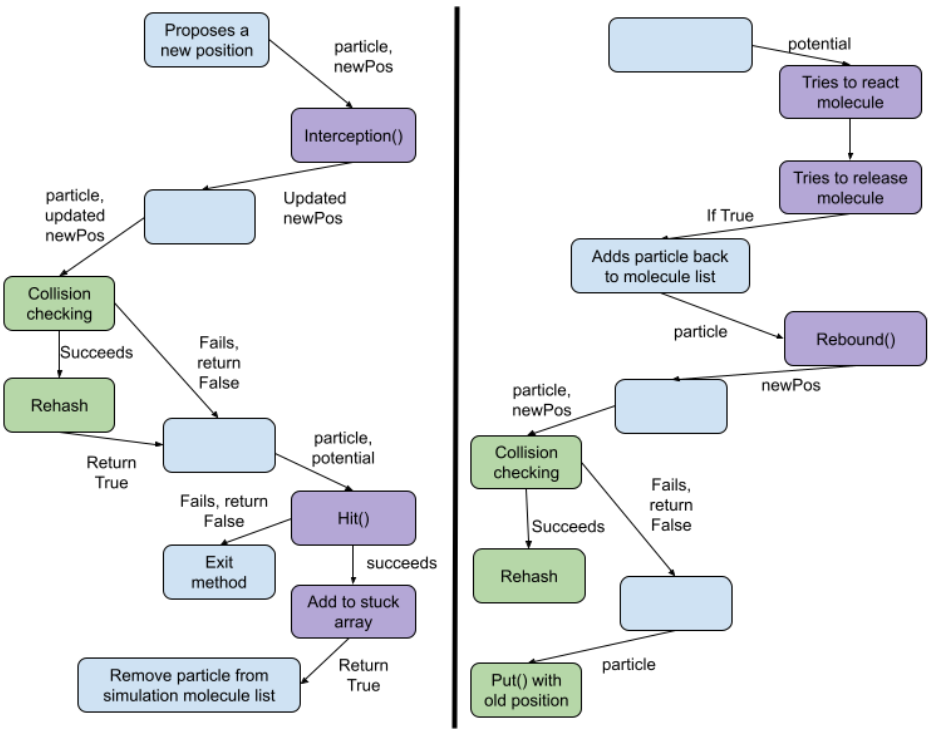

Figure 6.3: Flow chart between the outer simulation module (shown in blue), the electrode module (in purple), and the volume hash table (in green). The left side shows the movement routine for a particle originally free in the solution, the right side shows the routine for a particle beginning from the electrode `stuck` array.

Figure 6.3 shows a basic schematic of the flow between the three modules. [2] As can be seen, the main function of the outer simulation object primarily acts to interface between the electrode object and `volumeHT`.

The left side of the figure shows the routine for free particles in solution, which are kept in

[2] In reality, this is more complicated, as the outer simulation object also has to deal with boundary conditions; literal "corner cases". Also, there are in reality five modules interacting here, as the `molecule` object and `boxes` object are omitted for clarity.

a `molecules list` belonging to the simulation object. The object uses the normal distribution to propose a new position for the particle. If the particle is close to the electrode surface, the simulation object calls the electrode objects `interception()` method. If the new position does not collide with the surface, `inteception()` returns the proposed new position as it received it. If $\boldsymbol{p}_{\text{new}_z} <$ `particle.radius`, the method finds the interception coordinates and returns those to the outer simulation object. The simulation object then passes the particle object and the updated position proposal to the `volumeHT`. The `volumeHT` checks for collisions. If no collisions are identified, the particle's position is updated to be the new position. Then it is deleted from the `volumeHT` and reinserted so that it is saved in the appropriate box for its new position. If the collision fails, `volumeHT` returns `False` to the simulation object. The simulation object then calls the electrodes `hit()` method. `hit()` checks if the particle hits the surface. If it does not hit the surface, the routine is complete; we can exit `moveParticle()` and repeat for the next free particle. If the particle does hit the electrode, the electrode object adds it to its `stuck` array, and indicates that the simulation object should remove it from its `molecules` array, as it can no longer follow the displacement routine of a free particle.

Thr right side of Figure 6.3 shows the moving routine for a particle that begins in the electrodes `stuck` array. The electrode tries to react the molecule by calling the molecule object's `react()` method. The electrode then tries to release the molecule using the Bernoulli method described above. If the Bernoulli returns False, we can leave the particle in the stuck array and move on to the next particle. If it returns True, the electrode object passes the particle back to the simulation object for movement. The simulation object calls the electrode's `rebound()` function to propose a new position, which it passes to `volumeHT` to attempt to rehash the particle at the new position. If there is a collision, the simulation object rehashes the molecule at its original position. There should not be any collisions with the old position, because we know that the particle was able to take up that volume previously. Whether or not the displacement succeeds, the particle is returned to the simulation object's `molecule list`, so that it will be moved in the next iteration acccording to the free particle routine. In other words, even if the particle remains in contect with the electrode, it is no longer bound, and is not longer in the

stuck array.

6.3 Porous Electrode

The central aim of my project was to design a simulation that can account for specific electrode surface geometry, and in particular pores. In this light, I created a modified version of the electrode module, porousElectrode. In line with our focus on modularity, porousElectrode is designed so that it can be switched into the simulation object with minimal modification (see below). Its public methods are almost identical to its two-dimensional analogue, although it contains a larger number of initialization parameters and helper methods. In fact, porousElectrode actually relies on two helper classes, pore and areaHT.

6.3.1 The Pore Object

The pore object represents a singular pore. While the actual porous geometry of our chromium-gallium coated electrodes is unknown, we chose to use the simple model of a cylinder. This allows computational simplicty in providing a regular pore shape (although we allow pore radii to vary within one simulation) and the ability to decouple x and y coordinates from z. Because of this geometric decision, the pore object variables include its center coordinate, its radius, and its depth. In our chromium-gallium electrodes, the pore "walls" are insulating and therefore unreactive, while the glassy carbon "floor" allows electrons to flow. To account for this, we add boolean parameters floorReact and wallReact to our pore object. As boolean variables, these parameters can only be True or False; accounting for more continuous reactivity levels is a possible direction for future research. Because floorReact and wallReact are parameters, and not built into the simulation, these modules can easily be used to model conductive pores as well. The pore also takes in wallStick and floorStick parameters, that can differ from the surface stick parameter. Our lab has demonstrated that adding Nickel to the electrodes increases reactivity, and we have hypothesized that it does so by binding carbon dioxide molecules to the glassy carbon. Allowing flexibility in the stick

parameters allows for easily testing hypotheses such as this.

Unlike the flat electrode module, the `pore` object does not rely on the outer simulation object for moving its particles, nor does it rely on `volumeHT` for collision checking. In fact, the volume that is inside the pores is not volume represented by either `volumeHT`. This is because `volumeHT` does not have the information to know that particles in different pores cannot collide, and because the collision and stick conditions are more complicated than in the flat electrode case. This means that `pore` instances must take in the simulation's timestep as a parameter, and have their own collision checking methods and Gaussian displacement method.

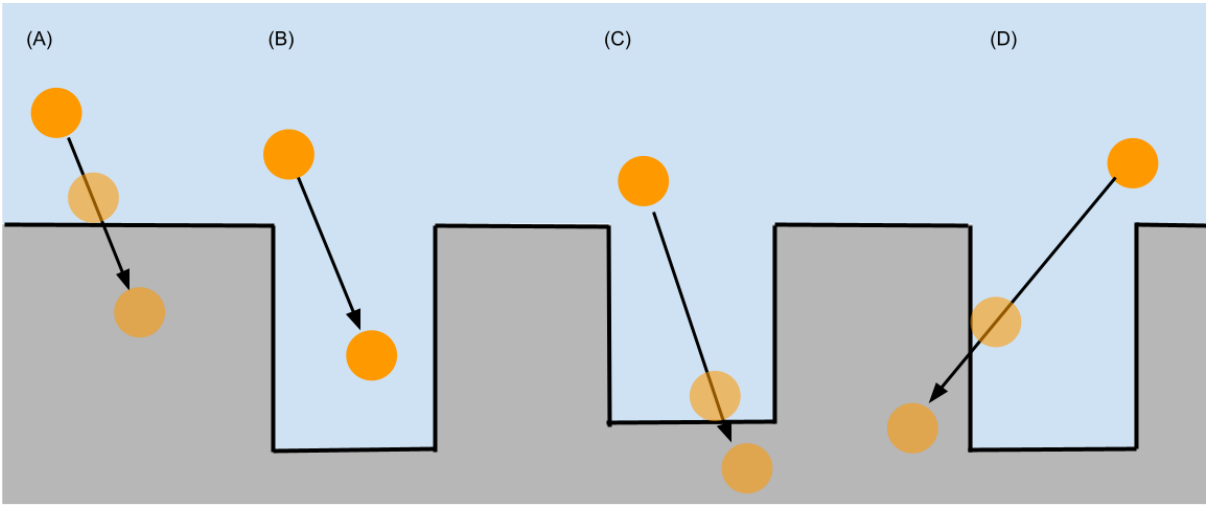

Figure 6.4: Four scenarios when a particle's z-coordinate is less than its radius. Starting from the left: a particle collides with the outer electrode surface, a particle is added to the interior of the pore, a particle collides with the pore floor, and a particle collides with the pore wall.

Pore Object: Geometry and Collisions

The core challenge of `pore` is capturing the geometry, and how it affects the interplay among the solution zone, outer electrode surface, pore walls, and pore floor. To start, we consider what happens in the `porousElectrode` case when a particle reaches the condition $\boldsymbol{p}_z <$ radius. Looking at Figure 6.4, we see that there are (at least) four distinct cases. The particle can collide with the outside surface, just as we described with the `interception()` method in the planar electrode case (scenario A). If the particle enters the pore, its new position might be in the interior of the pore volume (scenario B). It might also intercept with the pore floor (C) or pore walls. In order to know which `stick`, `react` and `rebound` conditions to

apply to a particle, we must differentiate between these two cases.

To do this, we use three geometric helper functions:

1. `xyInPore`(): This method is the simplest: it returns whether a (x, y) coordinate can be in the pore, accounting for the particle's radius. The condition it checks is:

$$(p_x - c_x)^2 + (p_y - c_y)^2 < (r_{\text{pore}} - r_{\text{particle}})^2$$

   where p_x is the particle's $x-$ coordinate, p_y is its $y-$ coordinate, c_x and c_y are the $x-$ and $y-$ coordinates of the pore's center, r_{pore} is the pore's radius, and r_{particle} is the particle's radius. (We assume that the particle's radius is always smaller than the pore's radius.) We must subtract the particle's radius from the pore's radius because it is not enough for the center of the particle to be in the pore; the entirety of its volume must fit. `xyInPore()` is a boolean method; it returns either `True` or `False`.

2. `posAtZ()`: This method is a generalization of the `interception()` method described above for the flat electrode. Whereas `interception()` took in a new position and an old position, and returned the coordinates at $z = r$, `posAtZ()` takes in an arbitrary z and returns the corresponding position along the particle's trajectory. The relevant equations are:

$$t = \frac{z - a_z}{b_z - a_z}$$
$$\boldsymbol{p} = \boldsymbol{a} + (\boldsymbol{b} - \boldsymbol{a})t$$

   where we use the same convention $\boldsymbol{a}$ = old position and $\boldsymbol{b}$ = new position as before. We made a seperate helper method `coordsFromT()` to perform the latter equation.

3. `xyIntercept()`: Like `posAtZ()`, `xyIntercept()` uses the parametric equation of a line to identify the parameter t that satisfies a relevant condition. In this case, we are looking to find t when the particle hits the pore's walls. At this point, we know that

$$(p_x - c_x)^2 + (p_y - c_y)^2 = (r_{\text{pore}} - r_{\text{particle}})^2$$

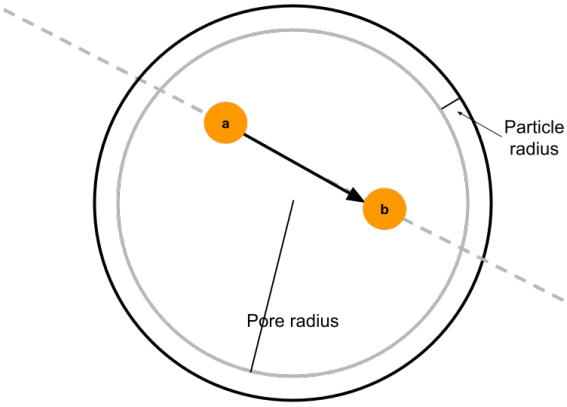

Figure 6.5: Finding the interception point of a particle's trajectory with the pore walls.

We plug in $p_x = a_x + (b_x - a_x)t$ and $p_y = a_y + (b_y - a_y)t$:

$$[(b_x - a_x)t + (a_x - c_x)]^2 + [(b_y - a_y)t + (a_y - c_y)]^2 = (r_{\text{pore}} - r_{\text{particle}})^2$$

This rearranges into a quadratic $At^2 + Bt + C = 0$, with

$$A = (b_x - a_x)^2 + (b_y - a_y)^2$$
$$B = 2[(b_x - a_x)(a_x - c_x) + (b_y - a_y)(a_y - c_y)]$$
$$C = (a_x - c_x)^2 + (a_y - c_y)^2 - (r_{\text{pore}} - r_{\text{particle}})^2$$

which we can plug directly into the quadratic formula $\frac{-B \pm \sqrt{B^2 - 4AC}}{2A}$. We claim that we can always take the $+\sqrt{B^2 - 4AC}$ value for t and that this value will always be real. To see this, see Figure 6.5. The two interception points of the dotted gray line, extending the line of the particle's trajectory, correspond to the two values of t. We only run this method if we know that the particle either began or ended in the pore or both. A line through a circle will always interact with a circle twice, unless it is a tangent line, in which case both roots will be equal and real, so taking the greater value of t still satisfies. The only other corner case is $(a_x, a_y) = (b_x - b_y)$, so that in the $xy - plane$, we are dealing with a dot not a line. However, since both coordinates are taken from a continuous distribtion,

this scenarios is sifficiently unlikely.

If the particle's final position is in the pore's interior (ie, more than a r_{particle} distance away from the walls), the interception point will be along the extension of the trajectory, not the trajectory itself. This corresponds to $t > 1$. Thus we check that $t \leq 1$.

Finally, we use the helper method `coordsFromT()` to use our value for t to determine the position.

With these helper methods, we are able to build `floorCollision()` and `wallCollision()` methods. `floorCollision()` first calls `posAtZ()` to obtain the $z-$ coordinate of the particle at $z = -\texttt{depth} + r_{\text{particle}}$. It then uses `xyInPore()` to check if the $x-$ and $y-$ positions are in the cylinder. If they are, then the particle collides with floor. The method then runs its own collision checker, which checks collisions with other particles in the pore, not the box. If it passes, its position is updated to be the output of `floorCollision()`, it is added to the `pore` object's `floorStuck` list, and discarded from its list of free particles.

`wallCollision()` works similarly, except that it uses `xyIntercept()` to check if the interception point is in an appropriate $z-$range: if the particle hits the circle boundary when its center is above -`depth` + r_{particle}, and no collisions are detected, the particle's position is updated, and it is then added to the `wallStuck` list and removed from the free particle list.

This provides a routine for placing particles within the pore. Once the object knows that the particle is within the pore, it can use `xyInPore()` to check if the particle is in the pore's interior, ie, not stuck to any surface. If so, it calls its collision checker. If no collisions are detected, the particle's position is updated, and then it is added to the free particle array. Otherwise, `floorCollision()` checks for collisions with the floor, and `wallCollision()` checks for collisions with walls.

The enumerated helper methods also render checking for entry and exit simple. Calling `posAtZ()` for $z = 0$ and feeding this result into `xyInPore()` determines whether entry/exit is possible.

Rebounding

Like the flat electrode module, the `pore` object requires `rebound` methods for displacing particles when they are released. For the floor, the same method can be used to find a trajectory. However, since the walls are curved and do not lie in the $xy-$ plane, they require a modified algorithm. The particles should be rebounding off the tangent plane to the cylinder, lying parallel to the $z-$axis. We accomplish this modification by starting with the original `rebound` method, and applying two linear transformations to the outgoing $\Delta\boldsymbol{p}$ vector.

The first linear transformation is a permutation. By permuting the (x, y, z) coordinates of the outgoing vector, we transform `rebound` to model particles bouncing off the $xz-$plane, ie, perpendicular to the $y-$ axis. (see Figure 6.6):

$$P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} z \\ x \\ y \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

We now need to rotate the outgoing displacement vector so that it aligns with movement away from the tangent plane of the particle's position $\boldsymbol{p}$. To do so, we begin by defining $\boldsymbol{d} \equiv \boldsymbol{p} - \boldsymbol{c}$. We use the $xy-$ rotation matrix R_θ, which rotates a vector θ radians counterclockwise about the $z-$ axis:

$$R_\theta = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

We now find $\sin\theta$ and $\cos\theta$ using trigonometry. Looking at the bottom of Figure 6.6, we see

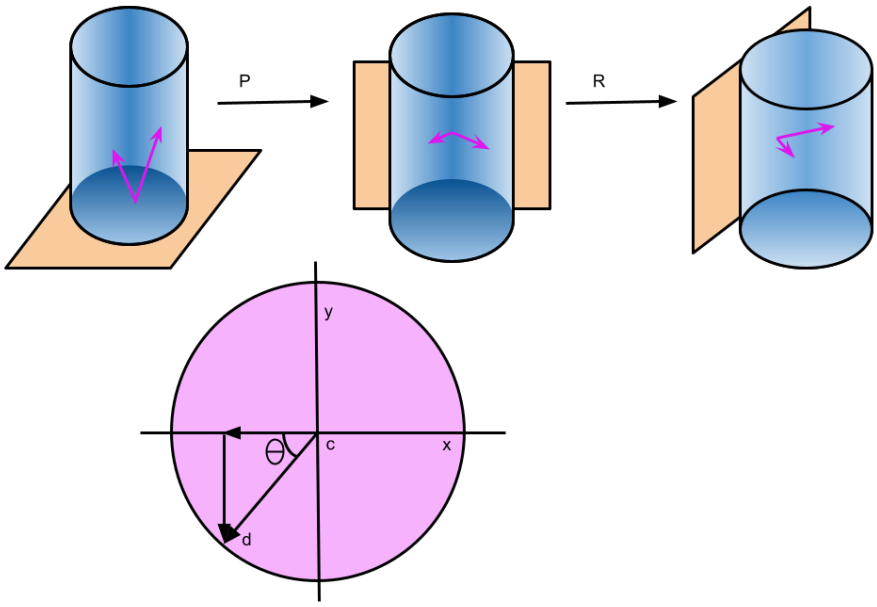

Figure 6.6: Top: Apply the permutation matrix, and then the rotation matrix, to find the rebound method for the tangent plane to the cylinder at position $\boldsymbol{p}$. Bottom: Use trigonometry to find the angle θ of the rotation matrix in terms of the position.

that we can apply

$$\sin\theta = \frac{\text{opposite}}{\text{hypotenuse}} = \frac{-d_y}{r_{\text{pore}} - r_{\text{particle}}}$$
$$\sin\theta = \frac{\text{adjacent}}{\text{hypotenuse}} = \frac{-d_x}{r_{\text{pore}} - r_{\text{particle}}}$$

We obtain hypotenuse = $r_{\text{pore}} - r_{\text{particle}}$ by recalling that we only use this method when a particle is adjacent to a wall. Plugging this in gives

$$R_\theta = \begin{bmatrix} \frac{-d_x}{r_{\text{pore}} - r_{\text{particle}}} & \frac{d_y}{r_{\text{pore}} - r_{\text{particle}}} & 0 \\ \frac{-d_y}{r_{\text{pore}} - r_{\text{particle}}} & \frac{-d_x}{r_{\text{pore}} - r_{\text{particle}}} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

We find the composition of these two transformations through multiplication:

$$R_\theta P = \begin{bmatrix} \frac{d_y}{r_{\text{pore}} - r_{\text{particle}}} & 0 & \frac{-d_x}{r_{\text{pore}} - r_{\text{particle}}} \\ \frac{-d_x}{r_{\text{pore}} - r_{\text{particle}}} & 0 & \frac{-d_y}{r_{\text{pore}} - r_{\text{particle}}} \\ 0 & 1 & 0 \end{bmatrix}$$

This means that a vector (x, y, z) gets mapped to

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} \rightarrow \begin{bmatrix} \frac{xd_y - zd_x}{r_{\text{pore}} - r_{\text{particle}}} \\ \frac{-xd_x - zd_y}{r_{\text{pore}} - r_{\text{particle}}} \\ y \end{bmatrix}$$

This supplies the displacement vector, $\Delta \boldsymbol{p}$, which is added to the original position.

6.3.2 The Area Hash Table

The next dependency of the `porousElectrode` module in `areaHT`. `areaHT` is similar to `volumeHT` in that it divides up space and uses a hash table for easy access. When a particle's $z-$coordinate falls below r_{particle}, the `porousElectrode` object must check if it has entered any pore. Naively, the `porousElectrode` object would have to call the entry checker method on each of its pore objects. We seek to use `volumeHT`'s compartmentalization of the space into boxes to increase efficiency. We call this method `areaHT` in recognition of the fact that unlike `volumeHT`, it only engages two dimensions–so the boxes are squares and the pore cylinders are circles.

The constructor of `areaHT` takes in an array of `pore` objects and three parameters from `volumeHT`: `sidelengths, spaceCoordinates,` and `numberOfBoxesPerDim`. It seeks to create a dictionary that has as its keys a `box` tuple and as its values a list of all the `pore` objects that overlap at all with that box. To do this, we draw the circumscribed square of the pore circle. We find its coordinates by obtaining the maximum and minimum points of the circle in the $x-$ and $y-$ dimensions: $\{c_x \pm r_{\text{pore}}, c_y \pm r_{\text{pore}}\}$. We then identify all the box squares within the circumscription square by using the `volumeHT` parameters to find the max

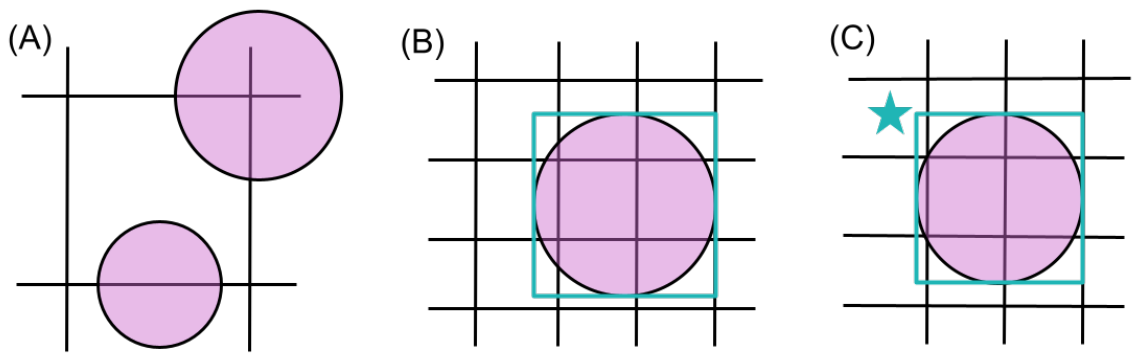

Figure 6.7: (A) A box can coincide with multiple pores. (B) use the circumscribed squared to idenitfy the coinciding boxes. (C) A corner case for the method.

and min box numbers in each direction, finding all the integers in these ranges, inclusive, and then taking the cartesian product of these two sets. At this point, we have found a map that takes in a pore circle and maps it to its boxes. We loop through to invert the function, generating a dictionary that serves as a map from a box to the pores with which it intersects.

This method does not work perfectly. Figure 6.7(C) shows a corner case, which is that the box denoted by the star is added as a box coinciding with the pore while in reality it does not. We leave this literal corner case alone, as it should not signifcantly detract from speed.

6.3.3 Interfacing with `porousElectrode`

`porousElectrode` is designed such that substituting it for its two-dimensional analogue in the simulation object requires few modifications. Largely, `porousElectrode` contains the same functionality as the flat electrode object, as well as abstractions that ease the interface bewteen the simulation object and the `pore` objects. The simulation object is given a boolean parameter `hasPores`. If `hasPores` is True, then the simulation object knows to make the following modifications to its routine.

The only modification in the simulation's code is in the `moveParticle()` method. After proposing a new position, the simulation object calls `porousElectrode`'s `entersPore()` method. This function begins by checking that the new positions $z-$ coordinate is less than zero. It then uses `areaHT` to get the box number (truncated, missing z) of the new position, and then uses the box number to return all the coinciding pores. For each pore, the

`porousElectrode` object checks if the new particle position is inside the pore and if the particle succeeded in passing through the pore opening. If this succeeds, `porousElectrode` uses the `pore`'s collision checker. This results in `porousElectrode` sending either a collision message or an entry message back to the simulation object. If the simulation receives an entry message, it removes the particle from the `volumeHT`, as the `pore` is now responsible for it. It then terminates the `moveParticle()` method. If the simulation object receives a collision message, it terminates the function and moves on to the next particle. If `porousElectrode` does not idenitfy a pore that the particle entered, it returns a `NOPORE` flag to the simulation, which proceeds to its behavior in the flat electrode case. (See Figure 6.8.)

Meanwhile, if the particle entered the pore, `porousElectrode` calls the pore's `entryPlacement()` method. This function checks if the particle is in the interior, intercepts the walls, or intercepts the floor, as described above. It takes care of updating the particle's position.

`porousElectrode` also allows the simulation to iterate through all particle attached to surfaces or in the pores with a single function call. When the simulation calls `movingAndShaking()`, `porousElectrode` calls `movingAndShakingPores()` as well as `movingAndShakingSurface()`, which is the same as the method for moving stuck particles in the flat electrode case. `movingAndShakingPores()` iterates through each pore in the object and for each one calls through instance methods: `movingAndShakingFloor()`, `movingAndShakingWalls()`, and `movingAndShakingInterior()`. The work is then moved to the `pore` object to attempt to move the particles. The first two of these methods attempts to react the particle, attempts to release the particles, proposes a new position with the rebound procedure, checks for collisions with other particles or surfaces, updates the particle's position, and checks if the particle has exited the pore. `movingAndShakingInterior()` proposes a new position with the 3-dimensional Gaussian procedure, checks for collisions, updates the position, and checks if the particle has exited the pore. The only thing passed back to the simulation object is a list of particles returned to the `volumeHT` from the pores; all this work is done internally, and the simulation object only has to care about exiting particles.

This routine also has a corner case: collision checking for exiting particles. The simulation object can access `pore`'s collision checker, so we can check if a particle collision will prevent particle entry. However, `pore` objects cannot access `volumeHT`'s collision checker, so exiting particles are not checked. This bug is part of the price payed for modularity, though future work could attempt to fix it.

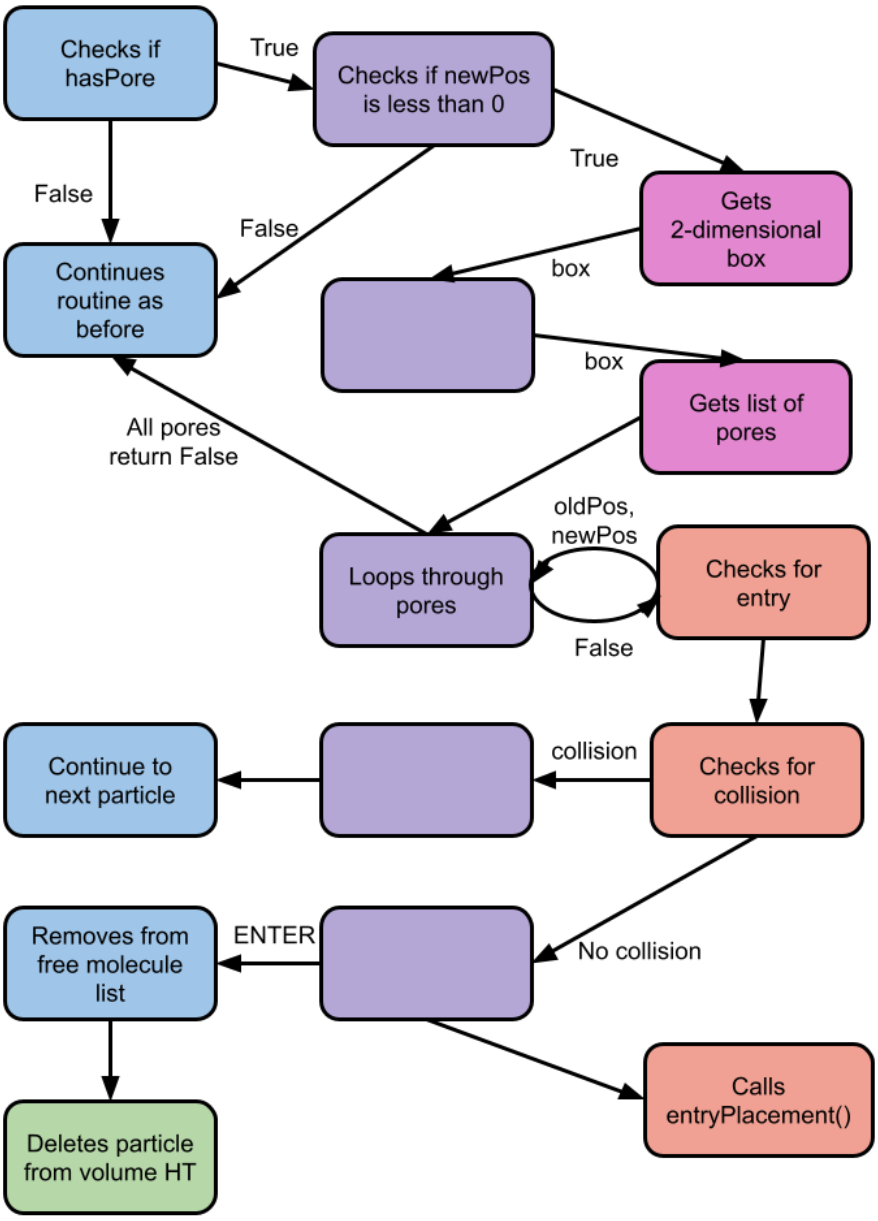

Figure 6.8: Flow chart that shows the interaction between modules in the porous version of `moveParticle()`. Blue indicates the outer simulation, green is `volumeHT`, purple is the `porousElectrode` object, pink is `areaHT`, and coral is the `pore` object.

Chapter 7

Conclusions & Future Work

7.1 Conclusions

In this study, we sought to build modularized and efficient Python objects for simulating CV's at electrodes with complex topologies. We did this through a simple diffusion-dominated kinematic paradaigm. We designed modules for (1) analyzing cyclic voltammograms, (2) baseline correcting, (3) representing chemical species, (4) dividing 3D space for efficient collision checking, (5) modelling electrodes, (6) modelling electrode pores, and finally (7) for simulating the data. Efficient implementation of each of these functions, in addition to the interactions among them, were carefully delineated. Code implementations were also drafted for each object; however, due to time constaints, not all were successfully debugged and assessed. The cyclic voltammogram libraries, which included visualization, analysis, and baseline correcting, were demonstrated to be successful on the sumple ferrocyanide system. However, we admit that utility in one specific reaction is not representative, nor does CV baseline correction present a novel result. The `volumeHT` was implemented and tested; while direct efficiency comparisons to other approaches remain a topic for future work, we have successful concluded its functionality. This accomplishment is significant, as `volumeHT` is the most general of our data structures and therefore has the most wide-ranging potential application.

Meanwhile, the electrode representation and simulation methods were not fully implemented and tested. While we did code implementations for these classes, verification of their

functionality and assessment of their accordance with real chemical systems remains for future work. Whether or not these particular implementations result in successful simulations, we have provided a framework for efficiently modeling pores. While we created an implementation for cylindrical pores, this maintains a large degree of flexibility on the client, who can pick the center and radius for each pore. Further, adapting pores to other geometries is straightforward as long as the $z-$ and (x, y) dimensions remain decoupled.

A large focus of this study was ensuring that our implementations were modular and efficienct. These goals were successfully implemented in our module designs. For example, by creating the `box` classes alongisde our `volumeHT`, we were able to obtain different functionality for the bulk and local regions without needing to alter the code in the simulation object. We were able to abstract away tasks such as collision checking and the `fromCenter` method. Similarly, for the electrode representations, the flat electrode and porous electrode modules could be inserted nearly identically in the simulation module. This allows for easy use and comparisons among experimental conditions. In the `porousElectrode` module, we were able to divide tasks between the electrode object and pore objects. This was another successful implementation of the principles of abstraction and modularization.

As for efficiency, we focused on achieving speed in collision checking. This involved constructing the `volumeHT` as well as the `fromCenter()` method to limit the total number of collision checking operations. In the porous electrode, we derived simple equations to check for collisions with the wall and floor as well as for entry/exit. We also developed an `areaHT` module to quickly search for relevant pores.

7.2 Future Work

As described above, the primary tasks left for future work is to complete the implementation of the electrode and simulation objects and to assess their efficacy in modelling cyclic voltammograms. If the model is shown to be successful in predicting experimental data, it can be used to assess the confinement effect hypothesis in the insulating pore case. The design of the data structures makes comparison to conducting pores straightforward.

Other future work can be used to directly evaluate the effiency gained by `volumeHT`. In chemical simulations, it is common practice to use a cutoff distance when assessing the potential energy interactions between a pair of particles. While this approach saves time in omitting calculations of insiginificant energy contributions, it still involves finding the normed distance with each particle in the solution. Comparing our three-dimension hash table approach with this cutoff approach could be a topic for future research.

Another research area is optimizing the density of the `volumeHT`. The constructor for `volumeHT` has a parameter `targetDensity`, the desired number of particles per box. Having smaller boxes is a trade-off: fewer particles per box means less particles to check for collisions; however, it also means fewer particles will be sufficiently close to the center in `fromCenter()`. Thus, `adjacent()` will need to be called more often. Finding the optimal balance point between these considerations could be a future endeavor.

Along with box sizes, this simulation can be used to optimize pore sizes for experiments. The electrode object allows assessing how long a particle remains in a pore once it enters, and how this relates to the ratios among pore depth, pore radius, and particle radius. This optimization can be used for creating more efficient chromium-gallium catalysts.

Finally, the collision checking methods can be converted into reaction pathways to model carbon dioxide reduction. As discussed earlier, it would be straightforward to replace the collision checkers in the code with a method that calls on the `molecule` object to react. The product would of course be determined by the second molecule; each chemical species would be a parameter in the other's function call. This modification would directly assess the primary purpose of this research: to develop chemical methods for helping reduce CO_2 to contribute to sustainability.

Chapter 8

References & Acknowledgements

8.1 References

1. Chen, C.; Khosrowabadi Kotyk, J. F.; Sheehan, S. W. Progress toward Commercial Application of Electrochemical Carbon Dioxide Reduction. Chem, 2018, 4, 2571–2586. https://doi.org/10.1016/j.chempr.2018.08.019.

2. Rieth, A. J.; Nocera, D. G. Hybrid Inorganic-Biological Systems: Faradaic and Quantum Efficiency, Necessary but Not Sufficient. Joule, 2020, 4, 2051–2055. https://doi.org/10.1016/j.joule.2020.08.012.

3. Torelli, D. A.; Francis, S. A.; Crompton, J. C.; Javier, A.; Thompson, J. R.; Brunschwig, B. S.; Soriaga, M. P.; Lewis, N. S. Nickel–Gallium-Catalyzed Electrochemical Reduction of CO2 to Highly Reduced Products at Low Overpotentials. ACS Catalysis, 2016, 6, 2100–2104. https://doi.org/10.1021/acscatal.5b02888.

4. Choi, M.; Bong, S.; Kim, J. W.; Lee, J. Formation of 1-Butanol from CO2 without *CO Dimerization on a Phosphorus-Rich Copper Cathode. ACS Energy Letters, 2021, 6, 2090–2095. https://doi.org/10.1021/acsenergylett.1c00723.

5. Calvinho, K. U. D.; Alherz, A. W.; Yap, K. M. K.; Laursen, A. B.; Hwang, S.; Bare, Z. J. L.; Clifford, Z.; Musgrave, C. B.; Dismukes, G. C. Surface Hydrides on Fe2P Electrocatalyst Reduce CO2 at Low Overpotential: Steering Selectivity to Ethylene Glycol. Journal

of the American Chemical Society, 2021, 143, 21275–21285. https://doi.org/10.1021/jacs.1c03428.

6. Zhou, Y.; Martín, A. J.; Dattila, F.; Xi, S.; López, N.; Pérez-Ramírez, J.; Yeo, B. S. Long-Chain Hydrocarbons by CO2 Electroreduction Using Polarized Nickel Catalysts. Nature Catalysis, 2022, 5, 545–554. https://doi.org/10.1038/s41929-022-00803-5.

7. Ting, L. R. L.; García-Muelas, R.; Martín, A. J.; Veenstra, F. L. P.; Chen, S. T.; Peng, Y.; Per, E. Y. X.; Pablo-García, S.; López, N.; Pérez-Ramírez, J.; Yeo, B. S. Electrochemical Reduction of Carbon Dioxide to 1-Butanol on Oxide-Derived Copper. Angewandte Chemie International Edition, 2020, 59, 21072–21079. https://doi.org/10.1002/anie.202008289.

8. Cronin, S. P.; Dulovic, S.; Lawrence, J. A.; Filsinger, K. A.; Hernandez-Gonzalez, A. P.; Evans, R.; Stiles, J. W.; Morris, J.; Pelczer, I.; Bocarsly, A. B. Direct Synthesis of 1-Butanol with High Faradaic Efficiency from CO2 Utilizing Cascade Catalysis at a Ni-Enhanced (Cr2O3)3Ga2O3 Electrocatalyst. Journal of the American Chemical Society, 2023, 145, 6762–6772. https://doi.org/10.1021/jacs.2c12251.

9. Lee, H. C.; Kim, J.; Yang, H. J.; Yu, J.; Bae, J. H. Electrochemical Reactions Affected by Electric Double Layer Overlap in Conducting Nanopores. Analytical Chemistry, 2024, 96, 18745–18753. https://doi.org/10.1021/acs.analchem.4c03956.

10. Edwards, M. A.; Robinson, D. A.; Ren, H.; Cheyne, C. G.; Tan, C. S.; White, H. S. Nanoscale Electrochemical Kinetics & Dynamics: The Challenges and Opportunities of Single-Entity Measurements. Faraday Discussions, 2018, 210, 9–28. https://doi.org/10.1039/c8fd00134k.

11. Sen, S.; Liu, D.; Palmore, G. T. R. Electrochemical Reduction of CO2 at Copper Nanofoams. ACS Catalysis, 2014, 4, 3091–3095. https://doi.org/10.1021/cs500522g.

12. Han, J.-H.; Lee, E.; Park, S.; Chang, R.; Chung, T. D. Effect of Nanoporous Structure on Enhanced Electrochemical Reaction. The Journal of Physical Chemistry C, 2010, 114, 9546–9553. https://doi.org/10.1021/jp909382b.

13. Bae,
J. H.; Han, J.-H.; Han, D.; Chung, T. D. Effects of Adsorption and Confinement on Nanoporous Electrochemistry. Faraday Discussions, 2013, 164, 361. https://doi.org/10.1039/c3fd00014a.

14. Seo, M.; Bae, J. H. Enhanced Electrocatalytic Activity by Nanoconfinement Effects at Nanoporous Indium Tin Oxide Electrodes. Bulletin of the Korean Chemical Society, 2024, 45, 993–999. https://doi.org/10.1002/bkcs.12912.

15. Fu, K.; Kwon, S.-R.; Han, D.; Bohn, P. W. Single Entity Electrochemistry in Nanopore Electrode Arrays: Ion Transport Meets Electron Transfer in Confined Geometries. Accounts of Chemical Research, 2020, 53, 719–728. https://doi.org/10.1021/acs.accounts.9b00543.

16. Turner, C. H.; Brennan, J. K.; Lísal, M.; Smith, W. R.; Johnson, J. K.; Gubbins, K. E. Simulation of Chemical Reaction Equilibria by the Reaction Ensemble Monte Carlo Method: A Review. Mol. Simul. 2008, 34 (2), 119–146. https://doi.org/10.1080/08927020801986564.

17. Bard, A. J.; Faulkner, L. R. Electrochemical Methods: Fundamentals and Applications; Wiley: New York, 2001.

8.2 Acknowledgements

I would like to first of all thank the Bocarsly Lab: Stephanie and Alma for their neverending kindness and support, Joseph for his poignant feedback, and Jacob for the comeraderie. Most of all, I would like to thank Professor Bocarsly for taking me on his lab late and giving me a project that so perfectly fits with my interdisciplanary scientific interests. I am so grateful for his laid back, positive attitude, and for his constant belief in me and my project.

I would like to thank Professor Jerelle Joseph for her incredible class on Molecular Modelling Methods, which provided the background necessary for this work. I am also grateful for her consultation on this project. I would like to thank Eli Yablon ('27) for helping me with mathematical components of this project.

I also want to thank the friends who have supported me through this time. Jenna Elliot ('25) and Madelaine LeBeau ('24), for their constant encouragement. Josh Schoenberg ('25), for always being kind and patient and supportive, for working side by side with me on my thesis for so many hours and listening to me complain about my struggles. Micah Jacobson ('28) and Jenny Davis ('28), for asking for progress. Davi Frank ('26), for encouraging me to care about the environment.

I would like to thank Princeton's chemistry department, in particular Dr. Susan VanderKam and Robert L'Esperance for thier advice and devotion to the senior thesis.

I would like to thank my family: my parents for their help and support, and my siblings for tolerating me reclusively reading an electrochemistry textbook during our South Africa trip. Finally, I would like to thank Hashem, for helping me through this experience and for the lessons it bequeathed upon me.